Subject: U24CST362 – Machine Learning Fundamentals

Course: B. Sc Computer Science with specialization in Artificial Intelligence and Machine Language

Prepared By: SAKSHI PANDEY

UNIT III

Binary Classification vs Multiclass Classification

Logistic Regression for Classification

Decision Tree for Classification and Regression

Random Forest Classifier and Regressor

Support Vector machine (SVM) - Linear and Kernel

Hyperparameter Tuning in Supervised Models

Confusion Matrix and Performance Metrics

Precision, Recall, F1-Score

ROC Curve and AUC Interpretation

Model Selection for Classification Tasks

CLASSIFICATION

Definition:

Classification is a supervised learning technique used to categorize data into predefined classes or labels based on input features.

How It Works:

- **Data Collection:** Gather labeled data (e.g., spam/not spam).
- **Feature Extraction:** Identify useful characteristics (color, texture, shape, etc.).
- **Model Training:** The model learns patterns between features and labels.
- **Evaluation:** Test performance on unseen data.
- **Prediction:** Predict class labels for new inputs.

Types of Classification:

- **Binary Classification:** Two categories (e.g., spam vs not spam).
- **Multiclass Classification:** More than two classes (e.g., cat, dog, bird).
- **Multi-label Classification:** One data point can belong to multiple classes (e.g., movie = action + comedy).

IN DETAIL

Common Algorithms:

- **Linear Models:** Logistic Regression, Linear SVM, Perceptron, SGD Classifier
- **Non-linear Models:** KNN, Kernel SVM, Naive Bayes, Decision Tree
- **Ensemble Models:** Random Forest, AdaBoost, Bagging, Voting, Extra Trees
- **Neural Networks:** Multi-layer Perceptrons

Key Concepts:

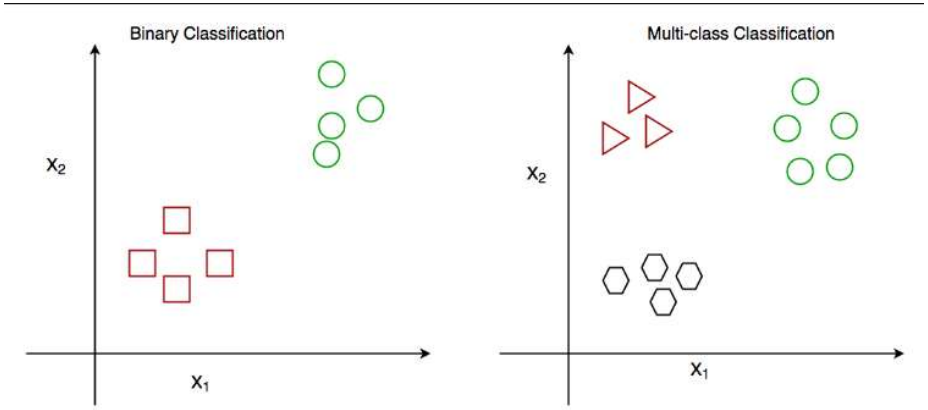
- **Decision Boundary:** Line/region separating classes.
- **Handling Imbalanced Data:** Use resampling or class weighting.
- **Interpretability:** Models like Decision Trees are easier to explain.

Real-life Applications:

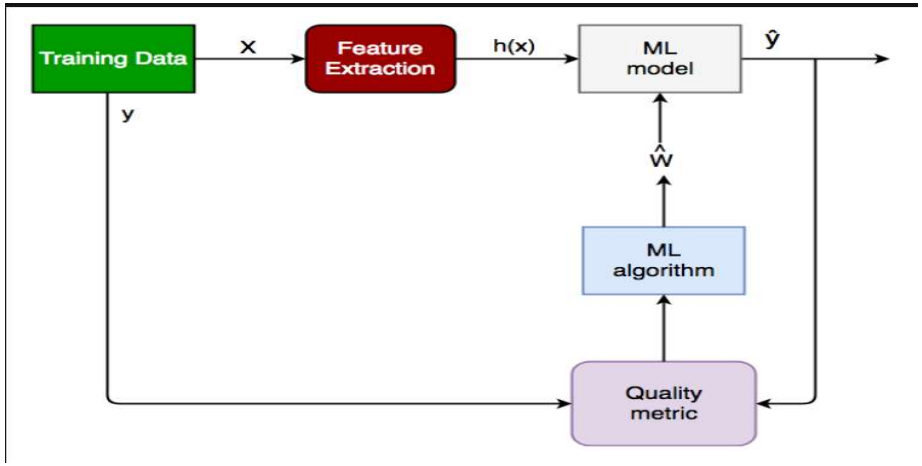
- Email Spam Filtering
- Credit Risk Assessment
- Medical Diagnosis
- Image Classification
- Sentiment Analysis
- Fraud Detection
- Recommendation Systems

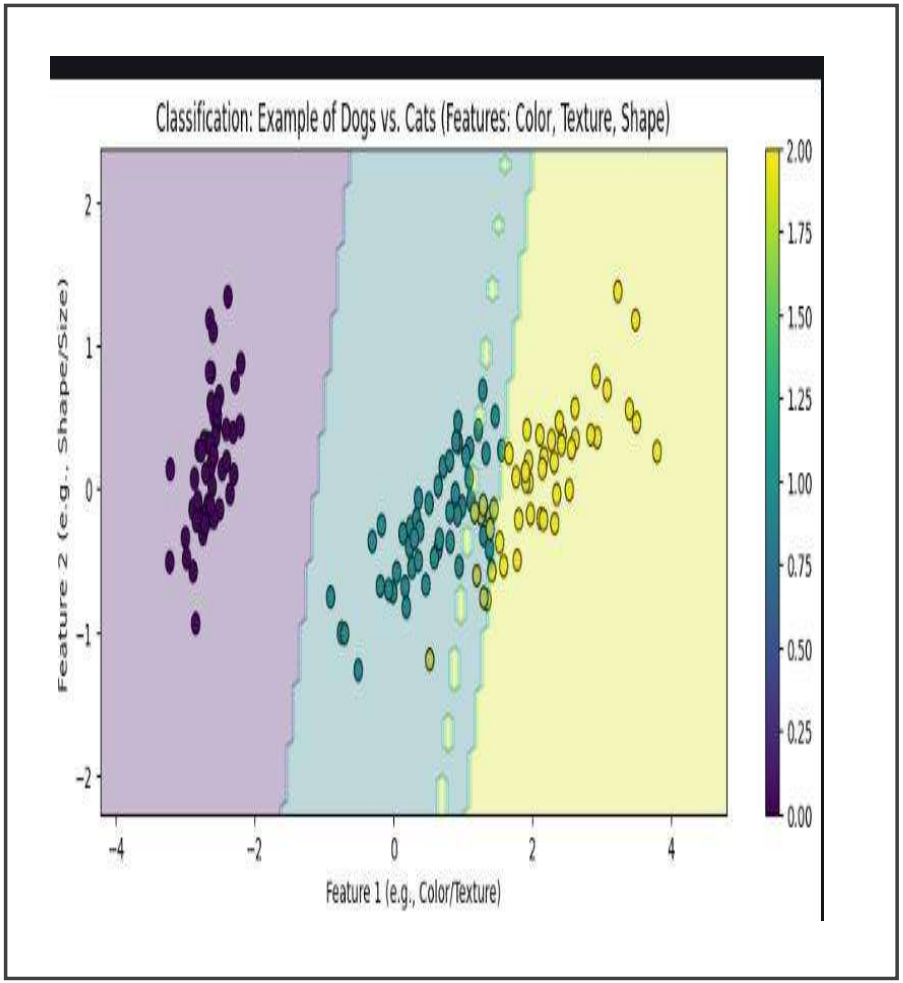
TYPES OF CLASSIFICATION

Type	Description	Example	Key Point
Binary Classification	Classifies data into **two categories** only.	Spam vs. Not Spam, Fraud vs. Legit	Simple Yes/No or True/False decision.
Multiclass Classification	Classifies data into **more than two classes**, but **only one label per input**.	Classifying animals as Cat, Dog, or Bird	Each data point belongs to **one** class only.
Multi-label Classification	Each data point can belong to **multiple classes simultaneously**.	A movie labeled as both *Action* and *Comedy*	Allows **multiple labels** for a single instance.



Binary classification vs Multi class classification





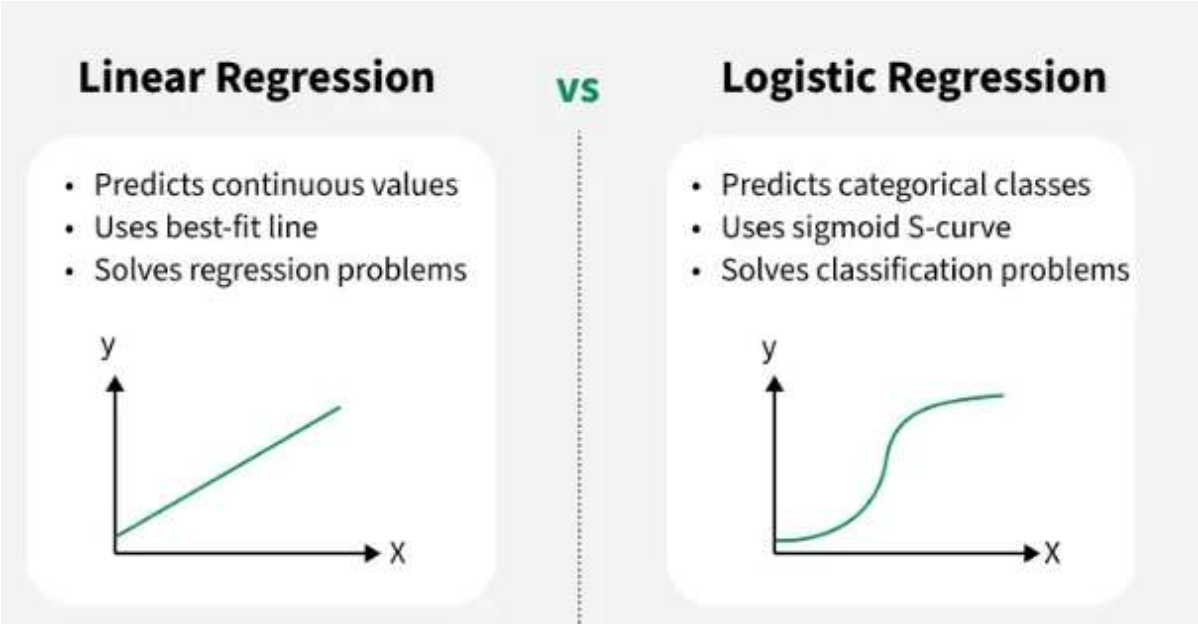
LOGISTIC REGRESSION

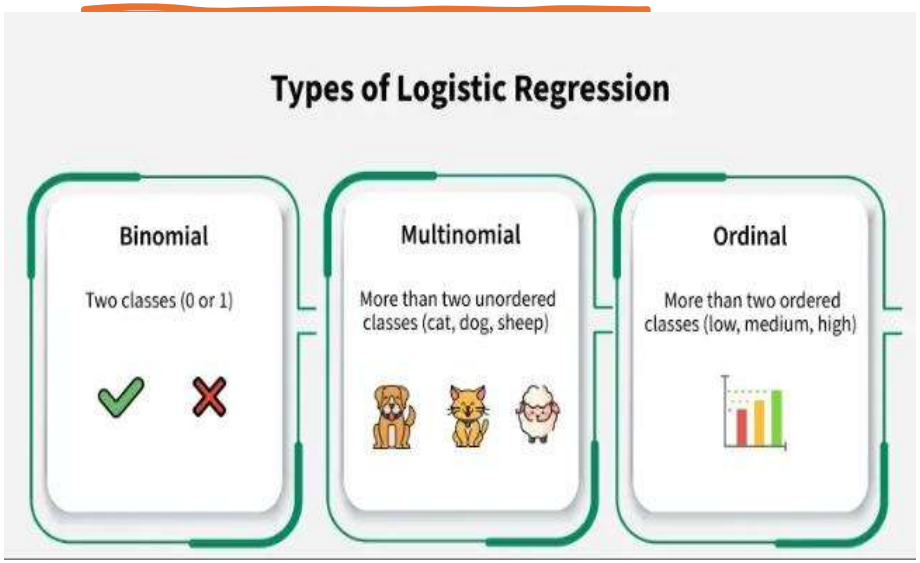
Introduction to Logistic Regression

Definition:

- Logistic Regression is a **supervised learning algorithm** used for **classification problems** (mainly binary).
- It predicts the **probability** that an input belongs to a specific class using the **sigmoid function**, which maps outputs between **0 and 1**.

FORMULA : $P(X)=1/1+e-(w\cdot X+b)$





Type	Description	Example
Binomial	Two possible outcomes	Yes/No, 0/1
Multinomial	Three or more unordered categories	Cat, Dog, Sheep
Ordinal	Ordered categories	Low, Medium, High

Working, Evaluation & Comparison

How It Works:

Compute linear combination:

Apply **Sigmoid Function** → Converts z into probability (0–1)

Classify using a threshold (e.g., ≥0.5 → Class 1, else Class 0)

Evaluation Metrics:

Accuracy = (TP + TN) / Total

Precision = TP / (TP + FP)

Recall = TP / (TP + FN)

F1 Score = 2 × (Precision × Recall) / (Precision + Recall)

AUC-ROC, AUC-PR – Overall performance across thresholds

Key Assumptions:

Independent observations

Linearity between features and log-odds

No extreme outliers

Large sample size

DIFFEREENCE

Linear Regression	**Logistic Regression**
Predicts **continuous** values	Predicts **categorical** values
Used for **regression** problems	Used for **classification** problems
Finds a **best-fit line**	Finds an **S-curve (Sigmoid)**
Uses **Least Squares**	Uses **Maximum Likelihood Estimation**

Logistic Regression and Sigmoid(Logit) Function

Logistic Regression is used when we need to predict the **probability** of an outcome, such as whether an email is spam or not spam. Instead of predicting continuous values like linear regression, it outputs values **between 0 and 1**, representing probabilities.

To achieve this, Logistic Regression uses the **Sigmoid (Logistic) Function**, defined as:f(x)

This function converts any real number into a value between **0 and 1**.
When the input x is a large positive number, the output is close to **1**.
When x is a large negative number, the output is close to **0**.
At x=0, the output is **0.5**.

The graph of the sigmoid function forms an **S-shaped curve**, smoothly mapping linear values to probabilities. This ensures that the model's predictions are always valid probabilities.

$$f(x) = \frac{1}{1 + e^{-x}}$$

where:

- $f(x)$ is the output of the sigmoid function.
- e is Euler's number: a mathematical constant ≈ 2.71828.
- x is the input to the sigmoid function.

Figure 1 shows the corresponding graph of the sigmoid function.

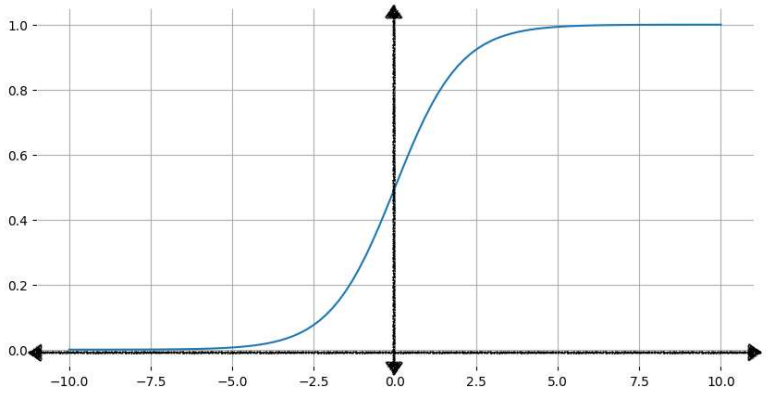


Figure 1. *Graph of the sigmoid function. The curve approaches 0 as x values decrease to negative infinity, and 1 as x values increase toward infinity.*

Logistic Regression Equation & Transformation

The Logistic Regression model first forms a **linear equation** using input features:z

Here, z is called the **log-odds**, since it represents the logarithm of the odds of the event happening.

Next, this linear output z is passed through the **sigmoid function**:y'

This transforms the raw linear score into a **probability** between 0 and 1.

If z is positive and large, the probability y' is close to 1.
If z is negative and large in magnitude, y' is close to 0.
When z=0, the model outputs a probability of 0.5, meaning it is equally likely to belong to either class.

In summary, Logistic Regression combines a **linear model** with a **sigmoid transformation**, turning numerical predictions into meaningful **probabilities** that can later be converted into binary decisions such as “Yes/No” or “True/False.”

$z = b + w_1x_1 + w_2x_2 + \ldots + w_Nx_N$, z is referred to as the *log-odds* because if you sta
y is the output of a logistic regression model, representing a probability):

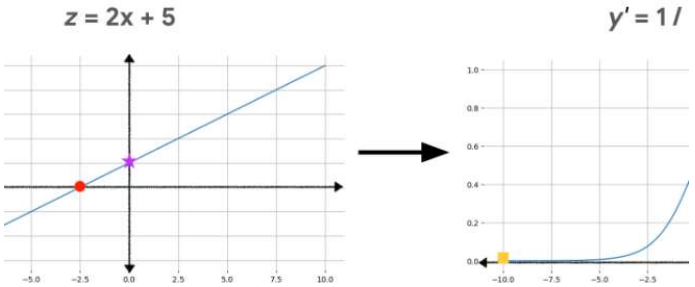
$$y = \frac{1}{1 + e^{-z}}$$

or z:

$$z = \ln\left(\frac{y}{1 - y}\right)$$

d as the natural logarithm of the ratio of the probabilities of the two possible outcomes: *y* and

es how linear output is transformed to logistic regression output using these calculations.



graph of the linear function z = 2x + 5, with three points highlighted. Right: Sigmoid curve with th after being transformed by the sigmoid function.

Probit function and Probit Regression

Problem with Linear Probability Model

- Linear models assume a straight-line relationship between predictors and probability.
- Predicted probabilities can fall outside **[0, 1]**, which is **not meaningful**.
- To fix this, we use **nonlinear models —** *Probit* and *Logit regression*.

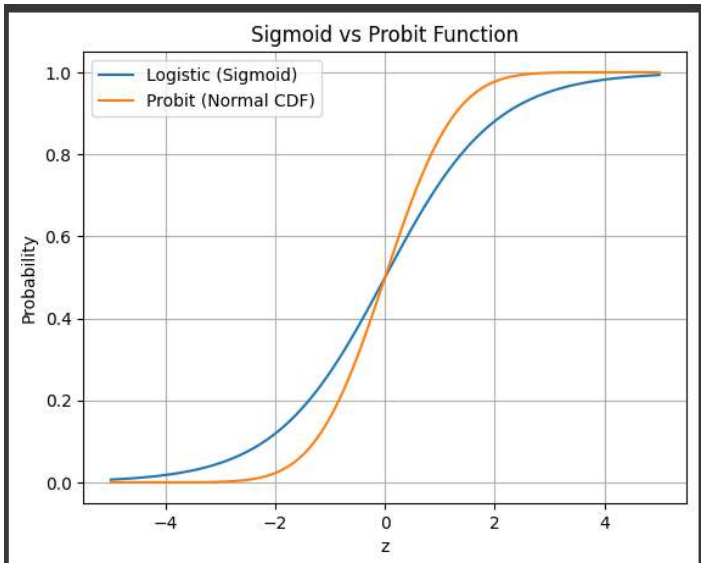
Probit Regression – Concept

- Uses **Cumulative Standard Normal Distribution (Φ)** to model probability.

  $P(Y=1|X)=\Phi(\beta 0+\beta 1X)$

  Here, $\beta 0+\beta 1X=z\rightarrow$ quantile of a standard normal variable.
- Relation between X and P(Y=1) is **nonlinear** (S-shaped curve).
- Ensures probabilities stay between **0 and 1**.
- Commonly used in **binary classification** problems (e.g., loan approval).

Probit function and Probit Regression



- **Key Points**
- Coefficients affect the latent variable zzz, not directly the probability.
- Predicted probability change can be found by:
    - Compute P(Y=1) for X.
    - ComputeP(Y=1) for X+ΔX
    - Take the difference.
- In R:
- glm(deny ~ pirat, family = binomial(link = "probit"), data = HMDA)

🔹 **Link Function:**

$$\text{probit}(p) = \Phi^{-1}(p)$$

where
Φ^{-1} = inverse of the **Cumulative Distribution Function (CDF)** of a **standard normal distribution**.

🔹 **Inverse (Sigmoid-like):**

$$p = \Phi(z)$$

where $\Phi(z)$ = CDF of standard normal distribution.

Logit Model Vs Probit Model

Features	Logit Model www.facebook.com/thesishelper01	Probit Model
Link Function	$\log\frac{\mu}{1+\mu}$	$\Phi^{-1}(\mu)$
Inverse Link Function	$\frac{e^{\eta}}{1+e^{\eta}}$	$\Phi(\eta)$
Conditional Probability Function (CDF)	$\Lambda(u) = \frac{1}{1+e^{-u}}$	$\Phi(u) = \int_{-\infty}^{u} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}\, dz$
Density	$g(z) = \frac{e^{z}}{1+e^{z}}$	$f(Z) = \frac{1}{\sqrt{2\pi}} \exp[-\frac{1}{2} Z^2]$
Mean (μ)	0	0
Variance (σ^2)	$\frac{\pi^2}{3}$	1
Distribution of the errors	$\ln(\frac{p_i}{1+p_i}) = \sum_{k=0}^{k=n} \beta_k x_{ik}$	$\Phi^{-1}(p_i) = \sum_{k=0}^{k=n} \beta_k x_{ik}$
Marginal Effect ($\partial p/\partial x_j$)	$\Lambda(x'\beta)[1-\Lambda(x'\beta)]\beta_j$	$\Phi(x'\beta)\,\beta_j$

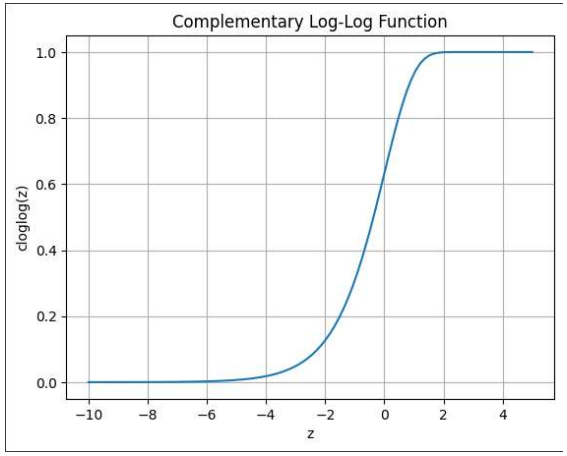
Logit Model Vs Probit Model

Features	Logit Model www.facebook.com/thesishelper01	Probit Model
Probabilities	The process of calculating probabilities in logit models are based on logistic functions that uses linear combinations.	The process of calculating probabilities in probit models are based on cumulative standard normal distribution function.
Linearity	Logit models are non-linear and provide predicted probabilities between 0 and 1.	Probit models are also non-linear and provide predicted probabilities between 0 and 1.
Estimation	Logit model cannot be estimated using ordinary least squares. Instead, maximum likelihood estimation is used, which requires assumptions about the distribution of the errors. Most often, the choice is between logistic errors which result in the logit model.	Probit model cannot be estimated using ordinary least squares. Instead, maximum likelihood estimation is used, which requires assumptions about the distribution of the errors. Most often, the choice is between normal errors which result in the probit model.
Selection	In practice many researchers select the logit model because of its comparative mathematical simplicity.	In practice many researchers do not select the probit model because of its comparative mathematical difficulty.
Nature	Simple Mathematics	Sophisticated Mathematics

Probit function vs Logit function

Complementary Log–Log (Cloglog) Function

- nother link function used in GLMs (especially when probabilities are *very close to 0 or 1*).
- Asymmetric curve — useful when “event” probabilities increase quickly then flatten out.



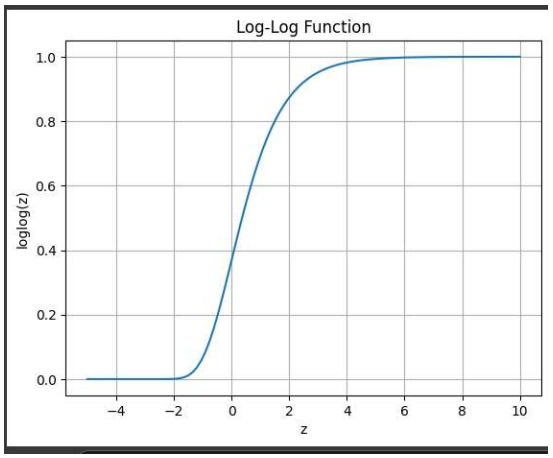
- **Link Function:**

$$\text{cloglog}(p) = \ln(-\ln(1-p))$$

- **Inverse:**

$$p = 1 - e^{-e^z}$$

Log–Log Function (Opposite of Cloglog)



- Used when high probabilities (near 1) are reached faster.

- **Link Function:**

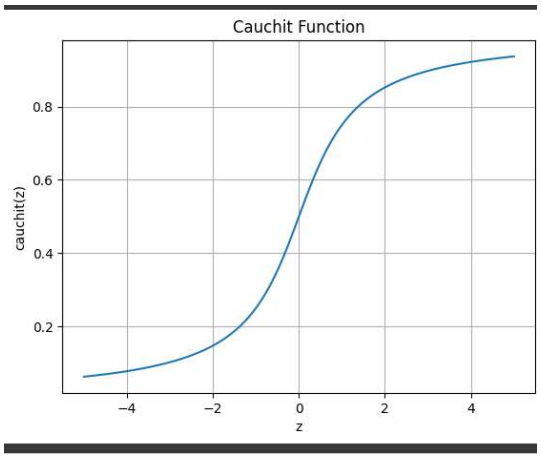
$$\text{loglog}(p) = -\ln(-\ln(p))$$

- **Inverse:**

$$p = e^{-e^{-z}}$$

Cauchit Function

- Based on the **Cauchy distribution** (heavier tails).
- Used when outliers are present (Cauchy distribution has heavier tails than normal/logistic).



- **Link Function:**

$$\text{cauchit}(p) = \tan\left[\pi(p - 0.5)\right]$$

- **Inverse:**

$$p = 0.5 + \frac{1}{\pi}\tan^{-1}(z)$$

Use cases

1. Sigmoid Function (Logit Link – Logistic Regression)

Use Cases:

- **Email Spam Detection –** Predicts whether an email is *spam (1)* or *not spam (0)*.
- **Medical Diagnosis –** Predicts probability of a disease (e.g., diabetes yes/no).
- **Credit Scoring –** Predicts loan approval probability.
- **Customer Churn Prediction –** Probability that a customer will leave a service.
- **Image Classification (Binary CNN output) –** Determines if an image belongs to a certain class or not.

Probit Function (Probit Link – Cumulative Normal Distribution)

Use Cases:

- **Toxicology Studies –** Probability of organism survival based on chemical dose.
- **Economics (Binary Choice Models) –** Predicts likelihood of *home loan approval / denial*.
- **Pharmacology –** Dose-response analysis (effective drug dose level).
- **Psychometrics –** Models binary responses in IQ or aptitude tests.
- **Political Science –** Probability a voter supports a candidate.

Use cases

Complementary Log-Log Function (cloglog Link)

Use Cases:

- **Survival Analysis –** Models event occurrence when probability increases sharply over time (e.g., machine failure).
- **Epidemiology –** Predicts infection risk over exposure time.
- **Reliability Engineering –** Probability of product failure under stress conditions.
- **Weather Forecasting –** Probability of extreme events (e.g., heavy rainfall).
- **Marketing Campaign Response –** Models rare but critical positive responses.

Log–Log Function (Log–Log Link)

Use Cases:

- **Extreme Value Analysis –** Probability of maximum/minimum temperature records.
- **Insurance Risk Modelling –** Estimating likelihood of rare, high-impact claims.
- **Hydrology –** Modelling extreme flood or drought probabilities.
- **Structural Engineering –** Predicts failure probability under stress load.
- **Environmental Science –** Estimation of pollution threshold exceedance.

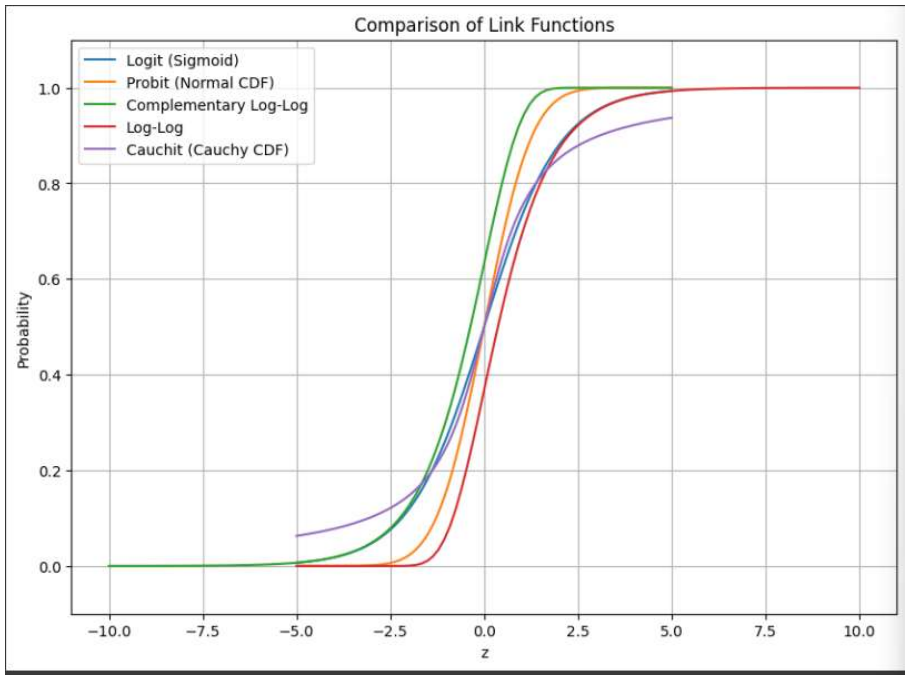
Use cases

- **Cauchit Function (Cauchy Link)**
- **Use Cases:**
- **Robust Binary Classification –** Handles datasets with extreme outliers.
- **Financial Risk Modelling –** Predicts market crash events with heavy-tailed data.
- **Astronomy –** Signal/noise classification when measurements have large variance.
- **Industrial Quality Control –** Detecting defective items with heavy-tailed error distributions.
- **Genetics –** Mutation probability modeling where distributions deviate from normal.

Function	Mathematical Form	Used In (Model Type)	Key Characteristics	Real-World Use Case
Sigmoid (Logit)	$f(x) = \frac{1}{1 + e^{-x}}$	Logistic Regression	S-shaped curve; outputs between 0–1	Email spam detection / Disease prediction
Probit	$f(x) = \Phi(x)$ where Φ = CDF of Normal(0,1)	Probit Regression	Similar to Sigmoid but assumes normal errors	Loan approval probability / Dose-response studies
Complementary Log-Log (cloglog)	$f(x) = 1 - e^{-e^{x}}$	Binary Regression (rare events)	Skewed S-shape; suitable for asymmetric data	Predicting machine failure / Infection risk
Log–Log	$f(x) = e^{-e^{-x}}$	Extreme Value Models	Opposite skew to cloglog; models min/max values	Flood or drought probability modeling
Cauchit	$f(x) = \frac{1}{\pi}\tan^{-1}(x) + 0.5$	Robust Regression	Heavy-tailed; resistant to outliers	Financial risk prediction / Outlier-prone datasets

Summary

Function	Link (Forward)	Inverse (to get Probability)	Common Use
Logit (Sigmoid)	log(p / (1-p))	1 / (1 + e^-z)	Logistic Regression
Probit	$\Phi^{-1}(p)$	$\Phi(z)$	Probit Regression
Cloglog	ln(-ln(1-p))	1 - e^-e^z	Event-time models
Loglog	-ln(-ln(p))	e^-e^-z	Reverse event-time models
Cauchit	$\tan[\pi(p - 0.5)]$	$0.5 + (1/\pi)\tan^{-1}(z)$	Heavy-tailed data



Measure of Dispersion

📉 2 Measures of Dispersion (Spread)

These functions describe how data is spread or varied.

Function	Formula / Meaning	Usage in ML
Variance (σ^2)	$\frac{\sum(x_i-\mu)^2}{n}$	Used in Gaussian models, feature scaling
Standard Deviation (σ)	√Variance	Measures spread in z-score normalization
Range	Max - Min	Basic spread measure
IQR (Interquartile Range)	Q3 − Q1	Used to detect outliers

Statistical Relationships

Function	Meaning	Use in ML
Covariance	Measures how two variables change together	Used in PCA, feature correlation
Correlation (r)	Standardized covariance, range -1 to 1	Used for feature selection, multicollinearity check
Chi-square (χ^2)	Tests independence between categorical variables	Used in feature selection for classification
Mutual Information	Measures dependency between variables	Feature selection in ML models

Measure of central tendency

These functions describe where the "center" of the data lies.

Function	Formula / Meaning	Usage in ML
Mean (μ)	Average value: $\frac{\sum x_i}{n}$	Used in normalization, loss calculation
Median	Middle value of sorted data	Robust to outliers (used in decision trees)
Mode	Most frequent value	Used in categorical data analysis, KNN classificati

Probability & Distribution Functions

Function	Description	Use in ML
Probability Density Function (PDF)	Likelihood of continuous random variable	Naïve Bayes, Gaussian models
Cumulative Distribution Function (CDF)	Probability ≤ x	Logistic regression, Probit models
Normal (Gaussian) Distribution	$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	Basis for many models (LDA, GNB)
Bernoulli/Binomial Distribution	Models binary outcomes	Logistic regression, classification
Poisson Distribution	Models count-based data	Event prediction models
Exponential Distribution	Time until next event	Survival analysis, reliability models

Statistical Estimation Functions

Function	Meaning	Use in ML
Maximum Likelihood Estimation (MLE)	Finds parameters that maximize data likelihood	Used in Logistic Regression, Naïve Bayes
MAP (Maximum A Posteriori)	Bayesian version of MLE	Used in Bayesian models
Confidence Interval	Range estimate for population parameters	Model evaluation
Hypothesis Testing (t-test, z-test)	Tests significance of relationships	Feature relevance checking

Error and Loss Functions

Statistically measure model performance.

Function	Formula / Meaning	Used In
Mean Squared Error (MSE)	$\frac{1}{n}\sum(y_i - \hat{y}_i)^2$	Linear Regression
Mean Absolute Error (MAE)	(\frac{1}{n}\sum	y_i - \hat{y}_i
Log Loss (Cross-Entropy)	$-[y\log(\hat{y}) + (1-y)\log(1-\hat{y})]$	Logistic Regression, Classification
R^2 Score (Coefficient of Determination)	$1 - \frac{SS_{res}}{SS_{tot}}$	Regression model evaluation

Advanced Statistical Functions in ML Algorithms

Function	Description	Example Algorithm
Z-score Normalization	$z = \frac{x-\mu}{\sigma}$	StandardScaler in sklearn
Entropy	$-\sum p(x) \log_2 p(x)$	Decision Trees, Information Gain
Gini Index	$1 - \sum p_i^2$	Decision Tree impurity measure
Likelihood Ratio	Ratio of model probabilities	Logistic Regression, Naïve Bayes
Bayes' Theorem	(P(A	B) = \frac{P(B
Mahalanobis Distance	Multivariate distance using covariance	Outlier detection, clustering
Kullback-Leibler Divergence (KL Divergence)	Difference between two probability distributions	Variational Autoencoders, Info theory

CART (Classification and Regression Tree)

- **Definition:**
- **CART (Classification and Regression Trees)** is a type of **Decision Tree algorithm** used for both **classification** and **regression** tasks.
- Developed by **Breiman, Friedman, Olshen, and Stone (1984)**.
- It recursively splits the dataset into smaller subsets based on feature values to make predictions.
- **Types of CART:**

- **Key Concepts:**
- **Tree Structure:** Consists of nodes (decisions), branches (splits), and leaves (outcomes).
- **Splitting Criteria:**
- *Classification* → Gini Impurity
- *Regression* → Residual Reduction

Type	Target Variable	Purpose
Classification Tree	Categorical	Predicts class labels (e.g., Spam / Not Spam)
Regression Tree	Continuous	Predicts numeric values (e.g., House Price)

WORKING

- **Working, Advantages & Applications**
- **Working Steps:**
- Find the **best split** using Gini Index (for classification).
- Split the data into **subsets** to increase purity.
- Repeat recursively until stopping criteria are met.
- **Prune** the tree to improve generalization.
- **Formula – Gini Impurity:**
- Gini=1−∑(pi)2
- where pip_ipi = probability of a class.
- 0 → Pure node, 1 → High impurity

- Advantages:
  - Handles both classification & regression
  - Simple, interpretable, and non-parametric
  - Performs implicit feature selection
  - Robust to outliers
- Limitations:
  - Can overfit on noisy data
  - Sensitive to small data changes
- Applications:
  - Credit risk analysis
  - Medical diagnosis
  - Environmental studies
  - Financial predictions

Decision Tree

Definition:
A **Decision Tree** is a tree-like model used for **classification and prediction**, representing decisions and their possible outcomes through nodes and branches.

Main Components:

Root Node: Represents the entire dataset.

Branches: Show decision paths.

Internal Nodes: Represent tests or decisions on features.

Leaf Nodes: Represent final outcomes or predictions.

Types of Decision Trees:

How It Works:

Start at root → ask feature-based questions.

Split data into subsets (Yes/No).

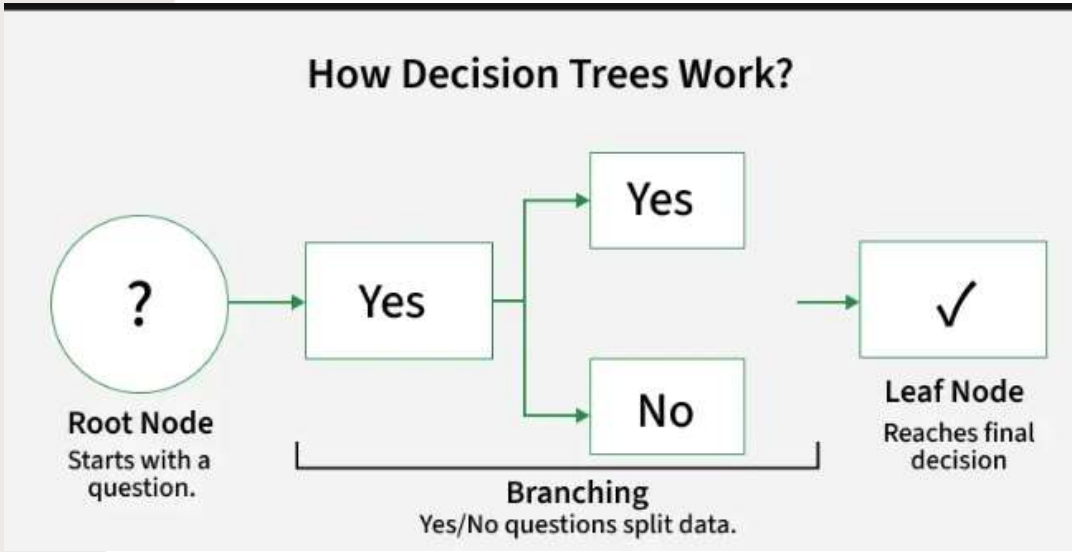
Continue splitting until reaching leaf nodes (final prediction).

Splitting Criteria:

Gini Impurity – measures purity of node.

Entropy – measures disorder or uncertainty.

Pruning:
Removes unnecessary branches to **prevent overfitting** and improve generalization.



Type	Purpose	Example
Classification Tree	Predicts **categorical** outcomes	Spam / Not Spam
Regression Tree	Predicts **continuous** outcomes	House Price Prediction

Random Forest

Definition:
Random Forest is an ensemble learning algorithm that builds **multiple Decision Trees** and combines their results for **better accuracy** and **reduced overfitting**.

Classification → Uses **majority voting**

Regression → Uses **average prediction**

Working Principle:

Create many decision trees using **random subsets** of data.

Each tree uses **random features** for splitting.

Each tree makes its own prediction.

Combine predictions → **Majority vote (classification)** or **Average (regression)**.

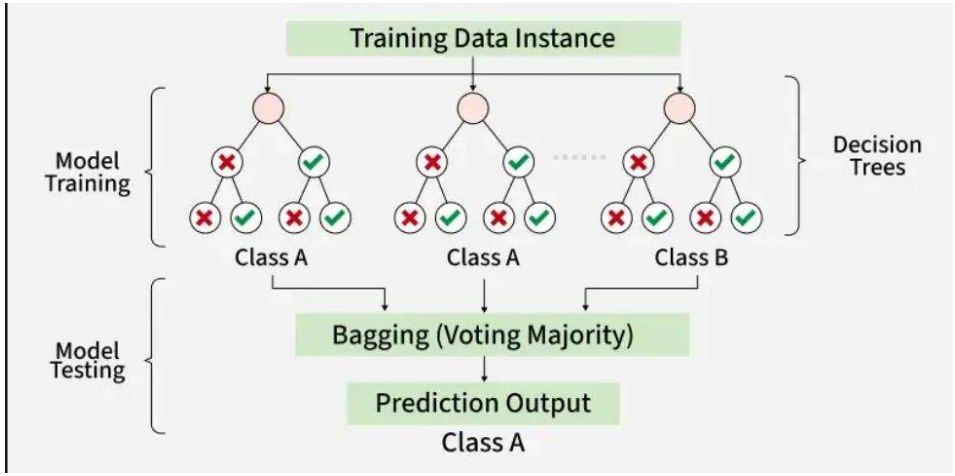
Key Features:

Handles **missing data**

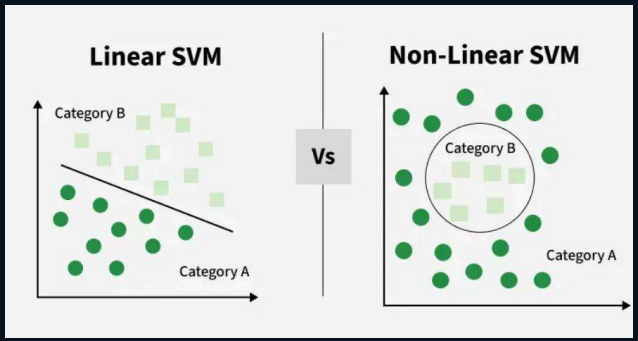
Shows **feature importance**

Works well with **large & complex datasets**

Suitable for both **classification & regression**



Support Vector Machines



Definition:

Support Vector Machine (SVM) is a supervised learning algorithm used for **classification and regression**. It finds the **best boundary (hyperplane)** that separates classes with the **maximum margin**.

Key Concepts:

- **Hyperplane:** Decision boundary separating classes.
- **Support Vectors:** Data points closest to the hyperplane that influence its position.
- **Margin:** Distance between hyperplane and support vectors (maximize this).
- **Kernel:** Maps non-linear data to higher dimensions for separation.
    - Linear, Polynomial, RBF (Gaussian).
- **Hard Margin:** No misclassification allowed.
- **Soft Margin:** Allows some errors for better generalization.

Working:

- Identify a hyperplane that best separates classes.
- Maximize margin between support vectors.
- Use kernel trick for non-linear data.
- Classify new points based on which side of hyperplane they fall.

Hyperparameter Tuning

- **Definition:**

Hyperparameter tuning is the process of finding the **best set of hyperparameters** that control the learning process of a machine learning model to achieve **optimal performance**.

- These are set **before training** (e.g., learning rate, number of trees, kernel type).
- Goal → **Improve accuracy**, **reduce overfitting**, and **enhance generalization**.

Key Techniques:

Advantages:
Boosts model performance

Prevents overfitting/underfitting

Improves generalization

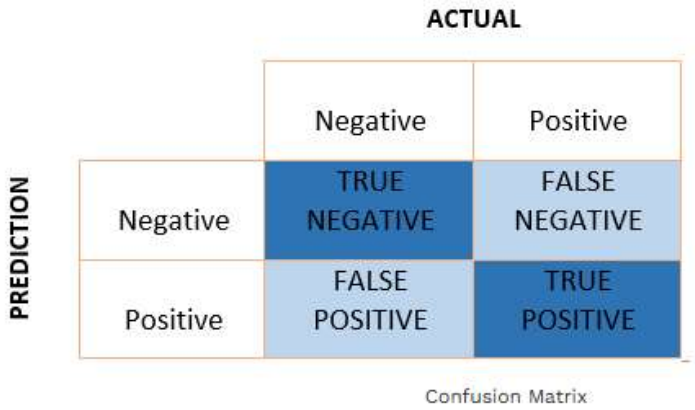
Optimizes resource usage

Challenges:
High computation cost
Large hyperparameter space
Requires domain knowledge

Applications:
Used in **tuning ML models** like SVM, Random Forest, Neural Networks, Logistic Regression for better results.

Method	Description	Pros / Cons
GridSearchCV	Tries **all combinations** of hyperparameters using cross-validation.	Accurate Slow & expensive
RandomizedSearchCV	Tries **random combinations** from a parameter range.	Faster May miss best combo
Bayesian Optimization	Learns from past results to **intelligently choose** next hyperparameters.	Efficient Complex to implement

Confusion Matrix



- **1. Confusion Matrix (2×2 for binary classification):**

- **TP:** Correct positive predictions
- **TN:** Correct negative predictions
- **FP (Type I Error):** Incorrect positive predictions
- **FN (Type II Error):** Incorrect negative predictions
- **2. Key Metrics:**
- **Precision:** TP / (TP + FP) → How many predicted positives are actually positive
- **Recall (Sensitivity / TPR):** TP / (TP + FN) → How many actual positives are correctly predicted
- **F1 Score:** Harmonic mean of Precision & Recall → Balances both, useful for imbalanced data
- **3. Metric Importance Examples:**
- **High Recall:** Medical diagnosis, fraud detection → minimize false negatives
- **High Precision:** Spam detection → minimize false positives
- **Takeaway:**

Confusion matrix & derived metrics give **better insight than accuracy**, especially for **imbalanced datasets** and critical applications.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

EVALUATION METRICS

Evaluation Metrics in Machine Learning

Purpose:
Measure how well a model performs in classification, regression, or clustering tasks to ensure optimal predictions.

Classification Metrics

- **Accuracy:** % of correct predictions; may mislead on imbalanced data.
- **Precision:** Correct positive predictions / All positive predictions; important when false positives are costly.
- **Recall (Sensitivity):** Correctly identified positives / All actual positives; crucial when missing positives is costly.
- **F1 Score:** Harmonic mean of Precision & Recall; balances both metrics.
- **Log Loss:** Penalizes low probability for true classes; lower is better.
- **AUC-ROC:** Measures model's ability to distinguish classes; higher AUC = better performance.
- **Confusion Matrix:** Shows TP, TN, FP, FN for detailed error analysis.

Regression Metrics

- **MAE (Mean Absolute Error):** Avg. absolute difference between predicted & actual.
- **MSE (Mean Squared Error):** Avg. squared difference; penalizes large errors.
- **RMSE:** Square root of MSE; same units as target.
- **RMSLE:** Penalizes underestimation; useful for wide-ranging targets.
- **R^2 (R-squared):** Proportion of variance explained by model; closer to 1 = better fit.

Clustering Metrics

- **Silhouette Score:** Measures cohesion & separation; closer to +1 = well-clustered.
- **Davies-Bouldin Index:** Measures cluster similarity; lower = better separation.

Key Takeaway:
Selecting the right metric for the problem type is essential to accurately evaluate and improve model performance.

ROC Curve and AUC

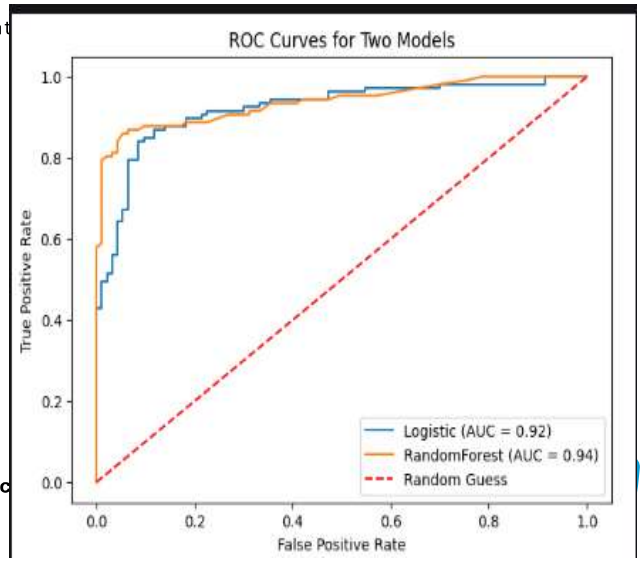
AUC-ROC Curve in Machine Learning

Purpose:

Evaluate how well a binary or multiclass classification model distinguishes between positive and negative classes across thresholds.

Key Components:

- **TPR (Recall/Sensitivity):** Correctly predicted positives / All actual positives
- **FPR:** Incorrectly predicted positives / All actual negatives
- **Specificity:** 1 – FPR
- **AUC (Area Under Curve):** Higher AUC → better class separat



ROC Curve:

- Plots TPR vs FPR at different thresholds
- Visualizes trade-off between sensitivity and specificity

Interpretation:

- **AUC = 1:** Perfect model
- **AUC ≈ 0.5:** Random guessing
- **AUC < 0.5:** Worse than random

Usage Tips:

- Effective for balanced datasets
- Less reliable for highly imbalanced datasets → consider **Prec**

Multiclass Extension:

- **One-vs-All approach:** Compute ROC & AUC for each class
- Combine curves to evaluate model's discrimination performance

Takeaway:

AUC-ROC provides a threshold-independent measure of model performance and its ability to rank positive cases higher than negative ones.

MODEL SELECTION

Model Selection in Machine Learning

Definition:
Choosing the best model for a task to ensure high accuracy, efficiency, and reliability.

Importance:

- Avoids **underfitting** (too simple) & **overfitting** (too complex)
- Optimizes **performance** and **computational efficiency**
- Ensures AI systems work well in **real-world scenarios**

Steps in Model Selection:

- **Understand Problem & Data:** Regression, Classification, Clustering; analyze dataset features.
- **Select Suitable Models:**
  - Regression → Linear Regression, Random Forest, Neural Networks
  - Classification → Logistic Regression, SVM, k-NN, Neural Networks
  - Clustering → k-Means, Hierarchical, DBSCAN
- **Model Evaluation:**
  - Split data: Training & Testing sets
  - Use **k-fold cross-validation**
  - Metrics: Accuracy, Precision, Recall, F1-score (Classification); MAE, MSE, R^2 (Regression)

Model Selection Techniques:

- **Grid Search:** Exhaustive hyperparameter search (computationally intensive)
- **Random Search:** Random subset of hyperparameters (faster)
- **Bayesian Optimization:** Uses probability models to predict best hyperparameters
- **Cross-Validation Based Selection:** Averages performance over multiple splits to avoid overfitting

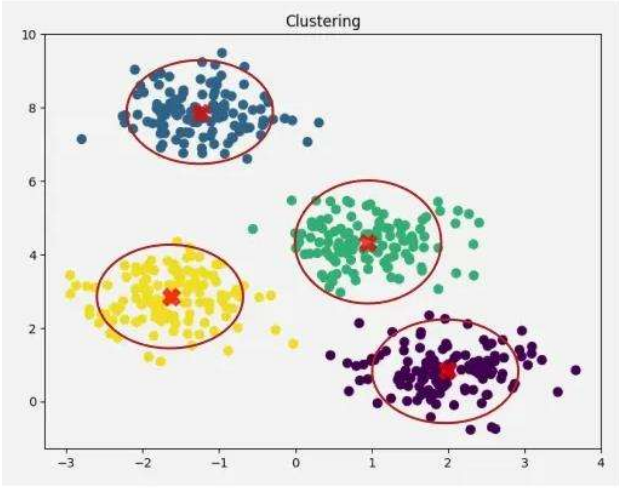
Takeaway:
Proper model selection ensures accurate predictions, **efficient computation**, and **robust real-world performance**.

Unit IV

- Concept of Clustering in ML
- K- Menas Clustering Algorithm
- DBSCAN Algorithm
- Hierarchical Clustering
- Evaluation of Clustering Models
- Introduction to Dimensionality Reduction
- Principal Component Analysis
- Linear Discriminant Analysis
- Feature Selection vs Feature Extraction
- Real- World Applications of Unsupervised Learning

Clustering

- **Definition:**
  Clustering is an *unsupervised learning* technique that groups similar data points into clusters based on similarity, without labeled data.
  The goal is to ensure that data points within the same cluster are more similar to each other than to those in different clusters.
- **Goal:**
  Discover natural patterns and hidden structures in data without predefined categories.
- **How It Works:**
  Data points are grouped using similarity or distance measures like *Euclidean* or *Cosine distance*.
- **Example:**
  In customer data, clustering can group buyers with similar shopping habits for targeted marketing or **personalized recommendations**.
- **Types of Clustering:**
- **Hard Clustering:** Each data point belongs strictly to one cluster.
  - Example: Customer belongs only to Cluster 1 or 2.
  - *Used for:* Market segmentation, document grouping.
- **Soft Clustering:** Data points can belong to multiple clusters with probabilities.
  - Example: 70% in Cluster 1, 30% in Cluster 2.
  - *Used for:* Customer personas, medical diagnosis.



Clustering

Clustering Methods & Applications

1. Centroid-Based (Partitioning)

- Algorithms: *K-Means, K-Medoids*
- **Pros:** Fast, scalable, simple
- **Cons:** Needs number of clusters, sensitive to outliers

2. Density-Based

- Algorithms: *DBSCAN, OPTICS*
- **Pros:** Detects arbitrary shapes, handles noise
- **Cons:** Parameter selection is difficult

3. Connectivity-Based (Hierarchical)

- Algorithms: *Agglomerative, Divisive*
- **Pros:** No need to predefine cluster count
- **Cons:** Computationally heavy

Recommendation Systems

Market Basket Analysis

4. Distribution-Based

- Algorithm: *Gaussian Mixture Model (GMM)*
- **Pros:** Flexible cluster shapes, probabilistic
- **Cons:** Needs number of components, sensitive to initialization

5. Fuzzy Clustering

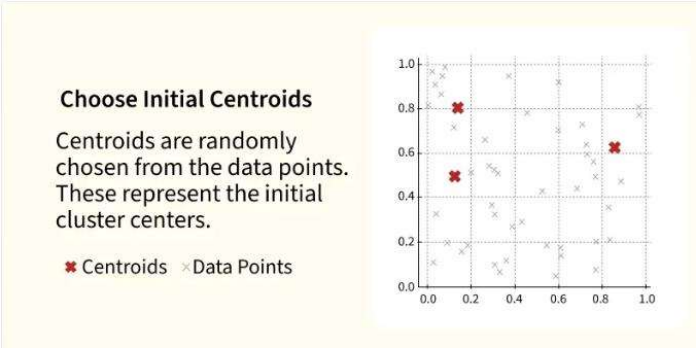
- Algorithm: *Fuzzy C-Means*
- **Pros:** Handles overlap, captures ambiguity
- **Cons:** More computationally expensive

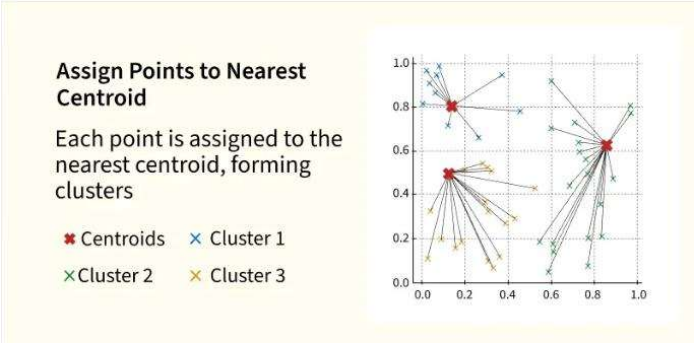
Use Cases:
Customer Segmentation **Anomaly Detection** **Image Segmentation**

Data Point	Hard Clustering	Soft Clustering
A	Cluster 1	Cluster 1: 0.91, Cluster 2: 0.09
B	Cluster 2	Cluster 1: 0.30, Cluster 2: 0.70
C	Cluster 3	Cluster 1: 0.17, Cluster 2: 0.83
D	Cluster 4	Cluster 1: 1.00, Cluster 2: 0.00

K-Means Clustering – Introduction & Working

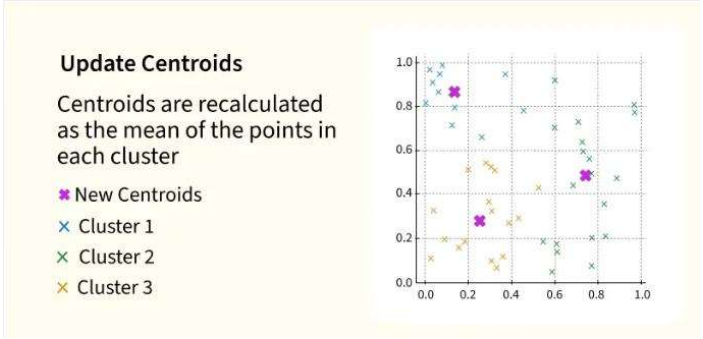
- **Definition:**
  K-Means is an unsupervised machine learning algorithm that groups unlabeled data into **k clusters** based on similarity. Each cluster is represented by its **centroid**, and data points are assigned to the nearest centroid.
- **Goal:**
  To partition data into clusters such that points within a cluster are more similar to each other than to those in other clusters.
- **Example:**
  An online store groups customers into segments like *"Budget Shoppers," "Frequent Buyers,"* and *"Big Spenders."*

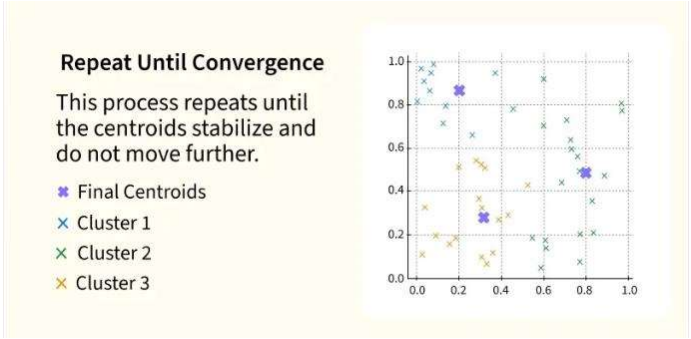




How K-Means Works:

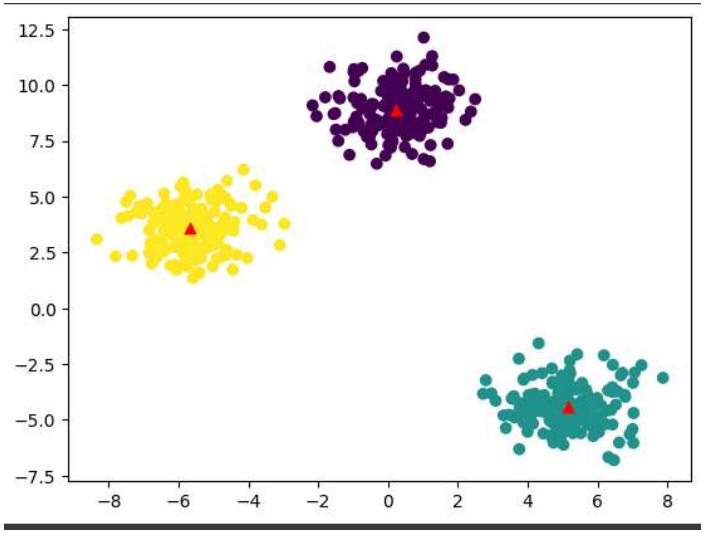
- **Initialization:** Randomly select *k* cluster centroids.
- **Assignment:** Assign each data point to its nearest centroid.
- **Update:** Recalculate centroids as mean of assigned points.
- **Repeat:** Continue until centroids stabilize.
- **Selecting Optimal K:**
  Use the Elbow Method — plot *Within Cluster Sum of Squares (WCSS)* vs *k* and find the point where adding clusters doesn't improve performance significantly.
- **Why Use K-Means?**
  Simple & Fast
  Effective for large datasets
  Widely used for segmentation, anomaly detection & image compression

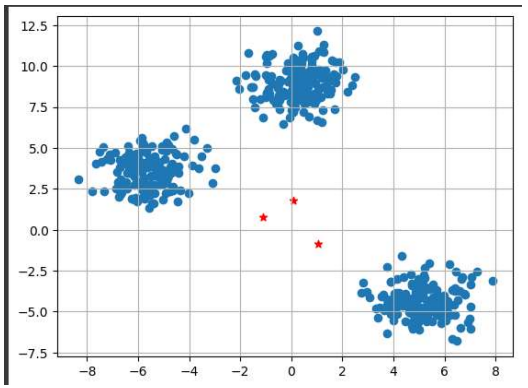




K Means Clustering - Applications

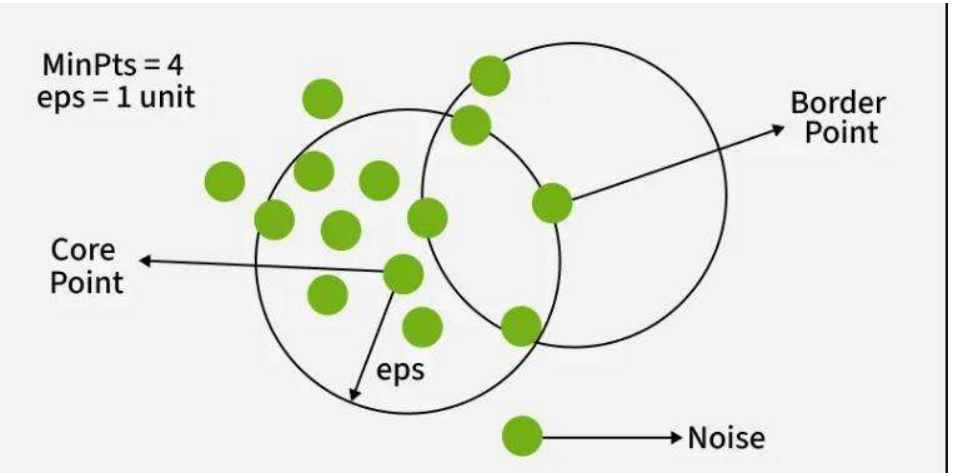
- **Applications:**
- **Customer Segmentation** – grouping users by purchase patterns
- **Image Compression** – reducing image colors using cluster centroids
- **Anomaly Detection** – identifying outliers that don't fit any cluster
- **Document Clustering** – grouping articles or news by similarity
- **Challenges:**
  Choosing the right number of clusters (*k*)
  Sensitive to initial centroid positions
  Struggles with non-spherical clusters
  Affected by outliers





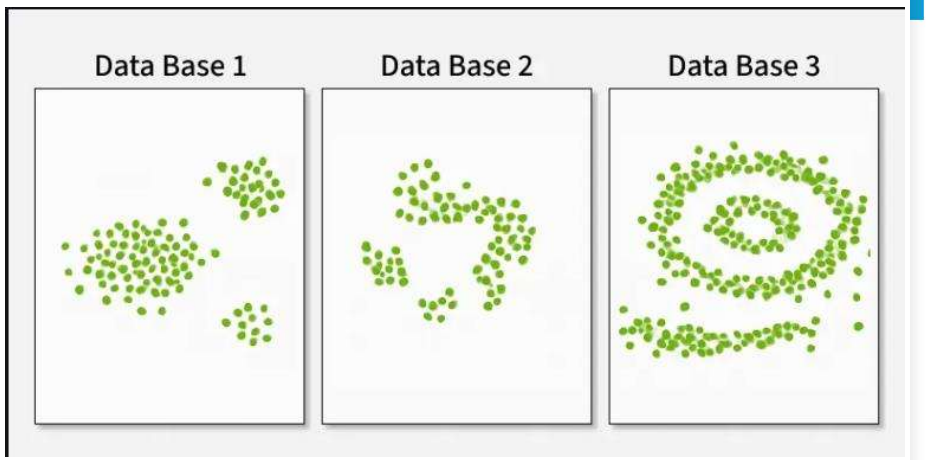
DBSCAN Clustering – Overview & Working

- **Definition:**
  DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is an **unsupervised clustering algorithm** that groups closely packed points and identifies outliers as noise.
  It is ideal for datasets with **arbitrary-shaped clusters** and **noise**.
- **Key Advantages:**
  Detects non-spherical clusters
  Handles noise and outliers effectively
  No need to predefine number of clusters (*unlike K-Means*)
- **Main Parameters:**
- **eps (ε):** Neighborhood radius around each point
- **MinPts:** Minimum number of points required to form a dense region
  (Rule: *MinPts ≥ D + 1*, where D = number of dimensions)



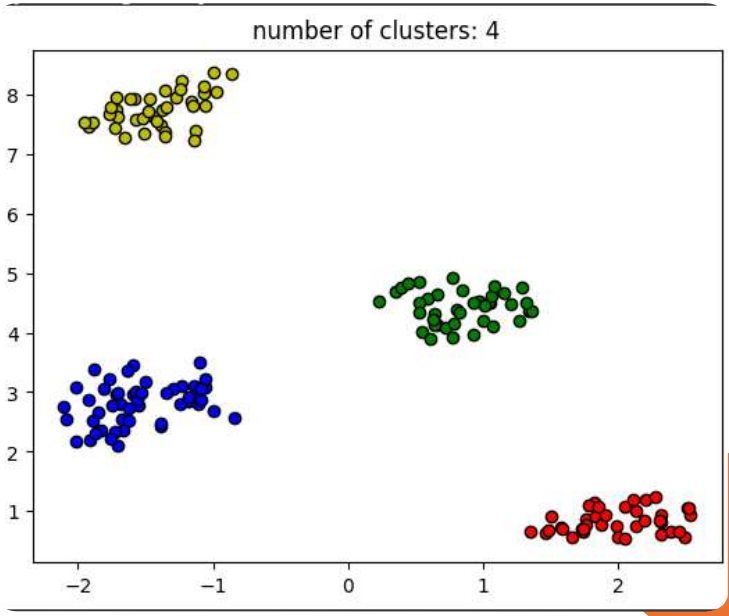
DBSCAN - Working

- **How DBSCAN Works:**
  Identify core points (dense regions with ≥ MinPts neighbors within ε).
- Connect density-reachable points to form clusters.
  Label remaining points as **noise**.
- **Types of Points:**
- **Core Points:** Have enough neighbors
- **Border Points:** Close to core points but not dense enough
- **Noise Points:** Do not belong to any cluster



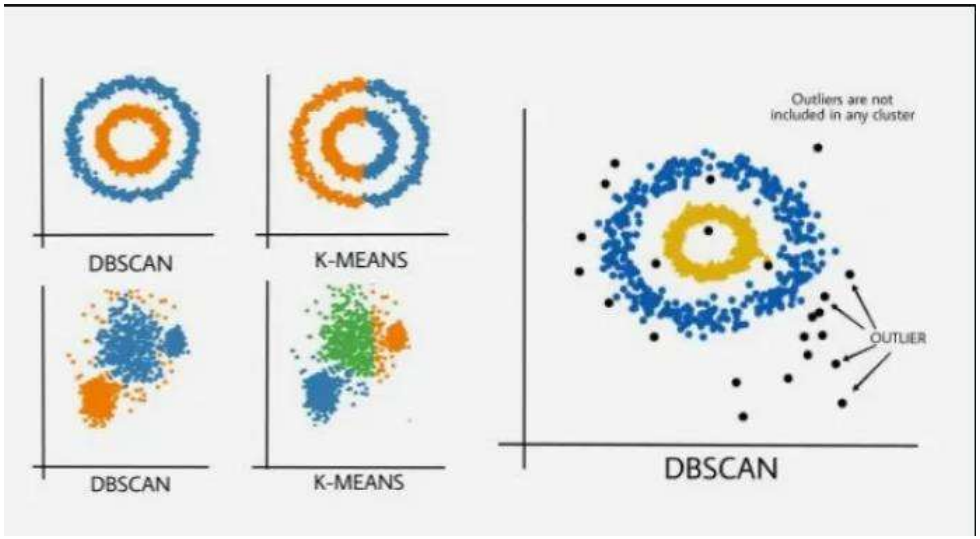
Implementation, Evaluation & Comparison

- **Evaluation Metrics:**
- **Silhouette Score (–1 to 1):** Measures cluster separation; closer to 1 is better.
- **Adjusted Rand Index (0 to 1):** Measures similarity between predicted and true labels.
- **DBSCAN vs K-Means:**
- DBSCAN does **not** require specifying number of clusters.
- Works well with **irregular shapes** and **noise**.
- K-Means assumes **spherical clusters** and is **sensitive to outliers**.
- **When to Use DBSCAN:**
- Unknown number of clusters
- Non-linear cluster shapes
- Data with outliers or variable density



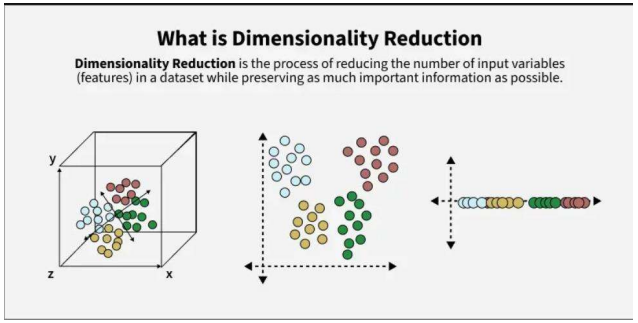
DBSCAN vs K Means

DBSCAN	K-Means
In DBSCAN we need not specify the number of clusters.	It is very sensitive to the number of clusters so it need to specified
Clusters formed in DBSCAN can be of any arbitrary shape.	Clusters formed are spherical or convex in shape
It can work well with datasets having noise and outliers	It does not work well with outliers data. Outliers can skew the clusters in K-Means to a very large extent.
In DBSCAN two parameters are required for training the Model	In K-Means only one parameter is required is for training the model



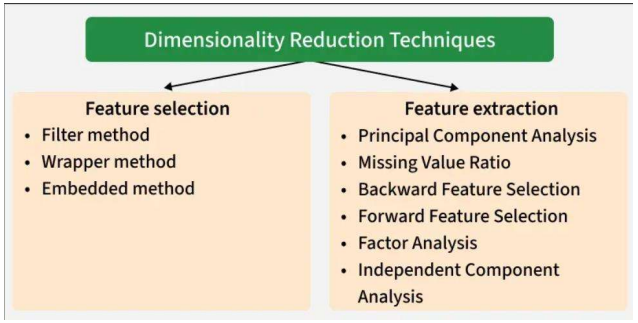
Introduction to Dimensionality Reduction

- **Definition:**
  Dimensionality Reduction is a technique to reduce the number of features in a dataset while retaining essential information.
  It simplifies high-dimensional data into fewer dimensions for **faster computation, easier visualization, and better model performance**.
- **Why It's Needed:**
- High-dimensional data → slow computation & risk of overfitting
- Reduces redundancy and noise
- Makes visualization & analysis easier
- **Example:**
  In predicting house prices, using too many features (like flooring, lighting, etc.) increases complexity.
  Reducing dimensions keeps only the most relevant ones (bedrooms, area, location).
- **How It Works:**
- Identifies which features carry the most variance/information
- Projects data from higher → lower dimensions while preserving structure
- **Benefits:**
  Faster computation
  Easier visualization
  Reduced overfitting
- **Limitation:**
  Some data loss & reduced accuracy possible



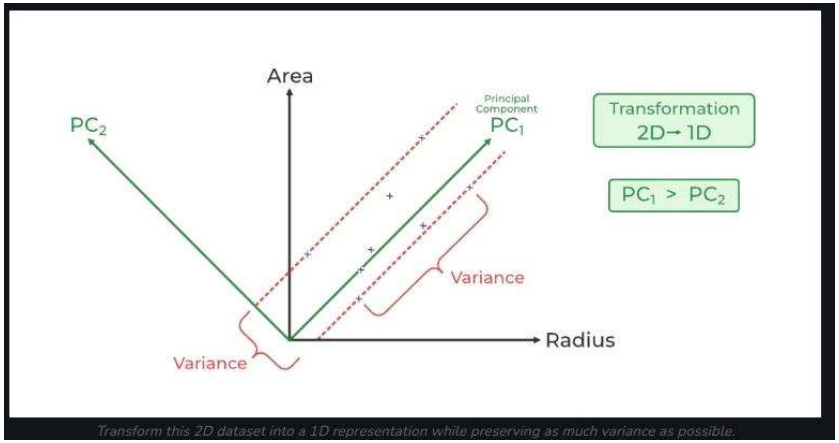
Techniques & Use Cases

- **Main Categories:**
- **Feature Selection –** Selects the most relevant original features
    - *Methods:* Filter, Wrapper, Embedded
    - *Examples:* Random Forest, Forward/Backward Selection
- **Feature Extraction –** Transforms data into new features
    - *Techniques:*
        - **PCA (Principal Component Analysis) –** Converts correlated variables to uncorrelated components
        - **Factor & Independent Component Analysis (ICA) –** Finds patterns or independent features
    - **Real-World Applications:**
- **Gene Expression Analysis:** Identify key genes faster
- **Text Categorization:** Reduce word features for classification
- **Image Retrieval:** Use fewer visual features for faster search
- **Intrusion Detection:** Select optimal features for cybersecurity models
- **Summary:**
  Dimensionality Reduction =
  Simpler models
  Faster learning
  Better generalization



Principal Component Analysis (PCA) – Overview

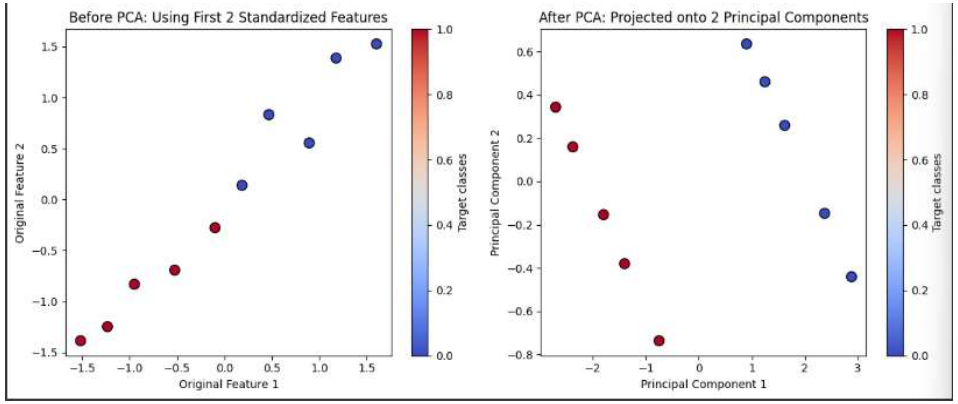
- **Definition:**
  PCA is a dimensionality reduction technique that transforms high-dimensional data into fewer variables (called *principal components*) while preserving most of the important information.
- **Purpose:**
- Simplifies complex datasets.
- Removes redundancy and correlation among features.
- Improves computational efficiency and visualization.
- **How PCA Works (Steps):**
- **Standardize the Data:** Convert all features to have mean = 0 and std = 1.
- **Compute Covariance Matrix:** Shows how variables vary together.
- **Find Eigenvalues & Eigenvectors:** Identify new axes (principal components) where data varies most.
- **Select Top Components:** Keep components that capture the highest variance.
- **Transform Data:** Project original data onto these new axes.
- **Key Concept:**
- **PC_1** captures the maximum variance (most information).
- **PC_2** is orthogonal to PC_1 and captures the next most variance.



Transform this 2D dataset into a 1D representation while preserving as much variance as possible.

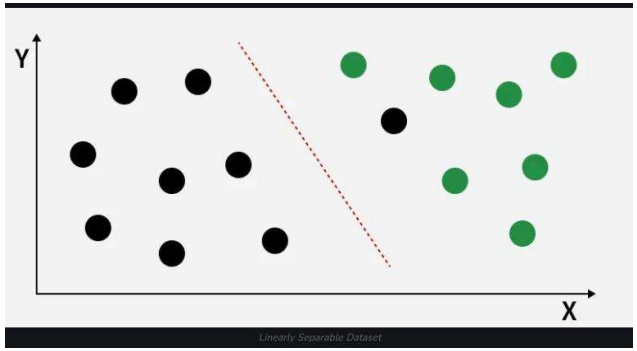
PCA Implementation

- **Dataset:**
  Features → Height, Weight, Age
  Target → Gender (1 = Male, 0 = Female)
- **Advantages:**
  Reduces dimensionality
  Removes multicollinearity
  Improves speed & visualization
- **Limitations:**
  Hard to interpret new features
  Some information loss possible



Linear Discriminant Analysis (LDA) – Overview

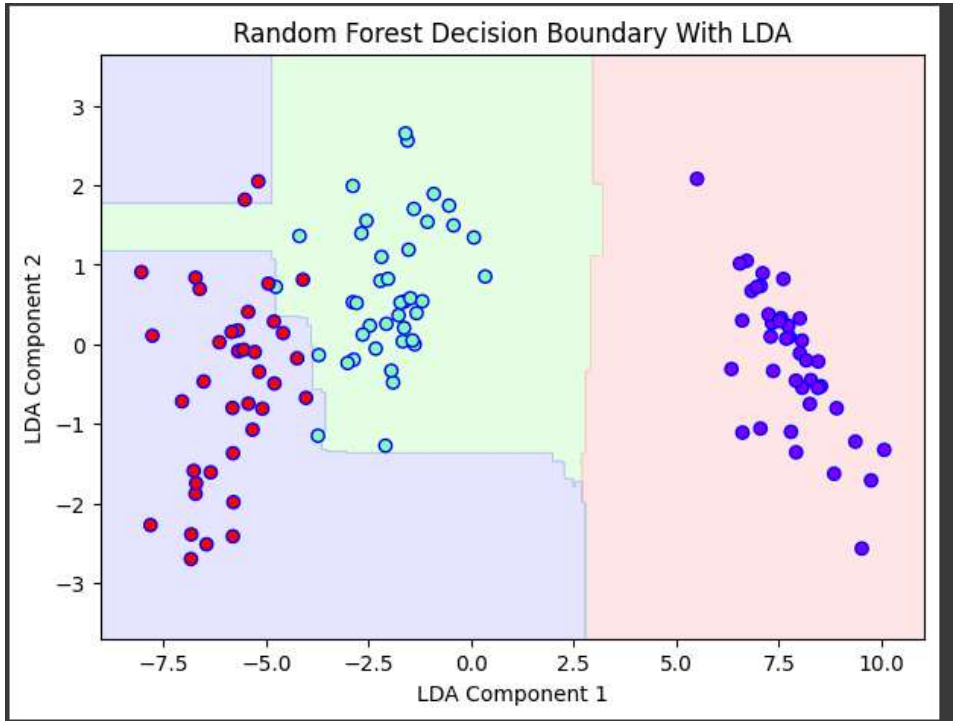
- **Definition:**
  Linear Discriminant Analysis (LDA) is a supervised dimensionality reduction and classification technique. It projects data onto a lower-dimensional space to **maximize class separability** while minimizing within-class variance.
- **Key Assumptions:**
- Data follows a **Gaussian (Normal)** distribution.
- **Equal covariance matrices** across classes.
- Data is **linearly separable** (can be divided by a straight line/plane).
- **Working Principle:**
- Compute **mean** and **scatter** (variance) for each class.
- Calculate **between-class scatter** and **within-class scatter** matrices.
- Find **eigenvectors and eigenvalues** from Sw−1Sb
- Select top components (directions) maximizing class separation.
- Project data onto new LDA axes.
- **Goal:**
  Maximize J(v)
- → Large numerator (between-class distance), small denominator (within-class spread).
- **Extensions:**
- **QDA:** Allows separate covariance per class.
- **FDA:** Handles non-linear separability.
- **RDA:** Adds regularization to avoid overfitting.



$$J(v) = \frac{|\mu_1 - \mu_2|^2}{s_1^2 + s_2^2}$$

LDA Implementation - Overview

- **Decision Boundary Visualization:**
- LDA compresses data to 2D space.
- Improves **class separability** compared to raw features.
- **Advantages:**
  Simple, fast, and interpretable.
  Handles multicollinearity well.
  Works well with small datasets.
- **Limitations:**
  Assumes normality & equal covariance.
  Fails on non-linear separability.
  Can lose information in high dimensions.
- **Applications:**
- Face recognition
- Medical diagnosis
- Customer segmentation



LDA vs PCA

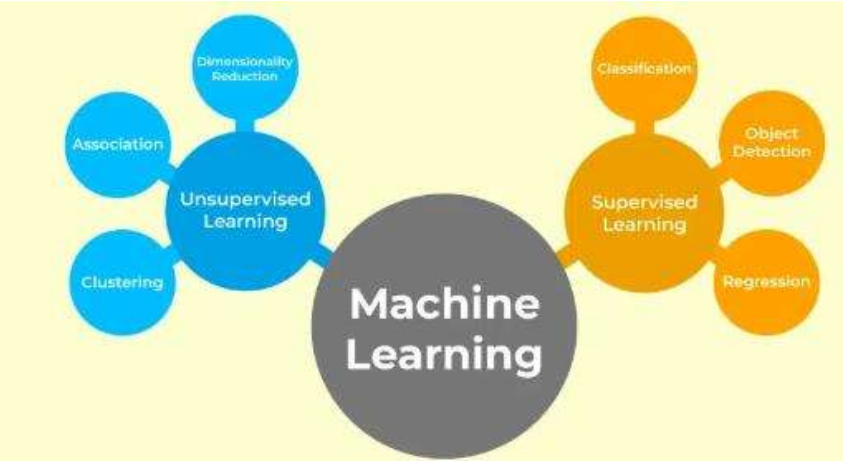
Basis	PCA (Principal Component Analysis)	LDA (Linear Discriminant Analysis)
Type	Unsupervised technique	Supervised technique
Objective	Maximizes **variance** in the data	Maximizes **class separability**
Input Requirement	Does **not require class labels**	Requires **class labels**
Focus	Finds directions of **maximum data spread**	Finds directions that **best separate classes**
Computation Based On	**Covariance matrix** of data	**Between-class** and **within-class scatter matrices**
Dimensionality Reduction	Can reduce to any number of components	Can reduce to **(C - 1)** components, where C = number of classes
Use Case	Feature extraction and noise reduction	Classification and pattern recognition
Data Distribution	No assumption about data distribution	Assumes **normal (Gaussian)** distribution and **equal covariance**
Outcome	Components are **uncorrelated**	Components are **discriminative**
Example Application	Image compression, visualization	Face recognition, medical diagnosis

Feature Selection vs Feature Extraction

Aspect	Feature Selection	Feature Extraction
Purpose	Selects a subset of relevant existing features	Creates new features from original data
Dimensionality Reduction	Keeps original features but reduces their number	Transforms data into a lower-dimensional space
Techniques Used	Filter, Wrapper, Embedded methods	PCA, LDA, Kernel PCA, Autoencoders
Data Requirement	Needs domain knowledge for selecting features	Can be applied to raw, high-dimensional data
Output Features	Original features (interpretable)	New transformed features (less interpretable)
Advantages	Enhances interpretability, reduces overfitting	Captures hidden patterns, handles nonlinearity
Limitations	May discard useful data	May introduce redundancy or noise

Real Life Examples of Unsupervised Learning

- **Customer Segmentation:**
  Businesses use unsupervised machine learning in **Python** to analyze user behavior, preferences, and purchase history. This helps identify distinct customer groups, allowing companies to design targeted marketing strategies, personalize offers, and improve product development. By segmenting customers based on data patterns, businesses can enhance engagement and satisfaction.
- **Anomaly Detection:**
  Unsupervised learning is applied to detect unusual patterns or outliers in datasets. This technique helps identify irregularities, such as fraudulent transactions or abnormal system behavior, improving security and performance monitoring.
- **Recommendation Systems:**
  Machine learning powers recommendation engines that personalize user experiences across platforms.
- Examples include **TED Talk**, **movie**, and **music** recommendations, where algorithms analyze preferences and suggest content aligned with user interests or emotions.
  These systems leverage data-driven insights to enhance engagement, satisfaction, and platform efficiency.



Unit V

- Steps to Build a Machine Learning Model
- Model Training and Testing Process
- Improving Model Accuracy and Generalization
- Random Forest Classifier
- Implementing K- Means and DBSCAN from Scratch
- Performance Metrics – Accuracy, Precision, Recall
- Senstivity and Specificity
- ROC Curve and AUC - Visualizattion
- Bias- Variance Tradeoff
- Model Comparison and Selection for Deployment

Steps to Build a Machine Learning Model

Machine Learning (ML) helps computers learn patterns from data and make intelligent predictions.
Building an ML model involves several systematic steps to ensure accuracy, efficiency, and reliability.

Key Steps:

Data Collection:
Gather relevant and high-quality data from sources like databases, APIs, or web scraping.
Good data quality improves the model's accuracy and generalization.

Data Preprocessing & Cleaning:
Transform raw data into usable form by removing missing or noisy values, encoding categorical variables, and scaling features.
Clean data ensures better training and performance.

Model Selection:
Choose the right algorithm based on the problem type:

- Regression → Predict continuous values
- Classification → Predict categories
- Clustering → Group similar data points

Model Training:
Feed preprocessed data into the chosen algorithm.
The model learns patterns by adjusting internal parameters using techniques like gradient descent.

Model Evaluation:
Assess performance using metrics such as:

- Regression → MAE, MSE, RMSE, R^2
- Classification → Accuracy, Precision, Recall, F1-score, AUC-ROC

Tuning & Optimization:
Improve model accuracy by adjusting hyperparameters using Grid Search, Random Search, or Cross-Validation.

Model Deployment:
Integrate the optimized model into a real-world application for live predictions.
Use tools like Docker and **Kubernetes** for efficient scaling and updates.

Conclusion:
ML model building is an iterative process— from collecting and cleaning data to deploying a tuned model that provides real-time, accurate predictions.

Training Set, Testing Set, Validation Set

Purpose of Data Splitting:

To evaluate how well a trained model generalizes to unseen data using the train_test_split() function from Scikit-learn.

Training Set:

- Used to **train** the model — it learns patterns and relationships from this data.
- Usually **60–70%** of total data.
- Quality and diversity of data directly affect model performance.
- Example:

  x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8)

→ 80% training, 20% testing.

Testing Set:

- Used **after training** to evaluate the model's performance.
- Usually **20–25%** of data.
- Independent from training data.
- Helps detect **overfitting** if accuracy on training > testing.
- Example:

  x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)

Set Type	Purpose	Data %	Used For
Training	Learn model parameters	60–70%	Model Training
Validation	Tune model	10–15%	Hyperparameter tuning
Testing	Evaluate model	20–25%	Final performance check

Validation Set:

- Used to **fine-tune hyperparameters** and assess model generalization.
- The model **does not learn** from this data — it's for evaluation only.
- Usually **10–15%** of total data.

Purpose:

- Helps avoid **overfitting** and improves **model tuning**.
- If validation accuracy > training accuracy → good generalization.

Splitting Example:

x_train, x_temp, y_train, y_temp = train_test_split(x, y, train_size=0.8)
x_val, x_test, y_val, y_test = train_test_split(x_temp, y_temp, test_size=0.5)

→ 80% Training | 10% Validation | 10% Testing

Implement Random Forest

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import
    RandomForestClassifier

from sklearn.metrics import accuracy_score,
    classification_report

import warnings

warnings.filterwarnings('ignore')

titanic_data = pd.read_csv('titanic.csv')

titanic_data =
    titanic_data.dropna(subset=['Survived'])

X = titanic_data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch',
    'Fare']]

y = titanic_data['Survived']

X.loc[:, 'Sex'] = X['Sex'].map({'female': 0, 'male': 1})

X.loc[:, 'Age'].fillna(X['Age'].median(), inplace=True)

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
```

```
rf_classifier = RandomForestClassifier(n_estimat
    ors=100, random_state=42)

rf_classifier.fit(X_train, y_train)

y_pred = rf_classifier.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

classification_rep = classification_report(y_test, y
    _pred)

print(f"Accuracy: {accuracy:.2f}")

print("\nClassification Report:\n", classification_r
    ep)

sample = X_test.iloc[0:1]

prediction = rf_classifier.predict(sample)

sample_dict = sample.iloc[0].to_dict()

print(f"\nSample Passenger: {sample_dict}")

print(f"Predicted Survival: {'Survived' if
prediction[0] == 1 else 'Did Not Survive'}")
```

Implement Random Forest For Regression

```
import pandas as pd

from sklearn.datasets import
    fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.ensemble import
    RandomForestRegressor

from sklearn.metrics import mean_squared_error,
    r2_score

california_housing = fetch_california_housing()

california_data =
    pd.DataFrame(california_housing.data,
    columns=california_housing.feature_names)

california_data['MEDV'] = california_housing.target

X = california_data.drop('MEDV', axis=1)

y = california_data['MEDV']
```

```
X_train, X_test, y_train, y_test = train_test_split(X,
y, test_size=0.2, random_state=42)

rf_regressor = RandomForestRegressor(n_estimat
ors=100, random_state=42)

rf_regressor.fit(X_train, y_train)

y_pred = rf_regressor.predict(X_test)

mse = mean_squared_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)

single_data = X_test.iloc[0].values.reshape(1, -1)

predicted_value = rf_regressor.predict(single_data
)

print(f"Predicted Value: {predicted_value[0]:.2f}")

print(f"Actual Value: {y_test.iloc[0]:.2f}")

print(f"Mean Squared Error: {mse:.2f}")

print(f"R-squared Score: {r2:.2f}")
```

Implement DBSCAN

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import normalize
from sklearn.decomposition import PCA
X = pd.read_csv('..input_path/CC_GENERAL.csv')
# Dropping the CUST_ID column from the data
X = X.drop('CUST_ID', axis = 1)
# Handling the missing values
X.fillna(method ='ffill', inplace = True)
print(X.head())
# Scaling the data to bring all the attributes to a comparable level
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

g

```
# Normalizing the data so that
# the data approximately follows a Gaussian distribution
X_normalized = normalize(X_scaled)
# Converting the numpy array into a pandas DataFrame
X_normalized = pd.DataFrame(X_normalized)

pca = PCA(n_components = 2)
X_principal = pca.fit_transform(X_normalized)
X_principal = pd.DataFrame(X_principal)
X_principal.columns = ['P1', 'P2']
print(X_principal.head())

# Numpy array of all the cluster labels assigned to each data
point
db_default = DBSCAN(eps = 0.0375, min_samples =
3).fit(X_principal)
labels = db_default.labels_

db = DBSCAN(eps = 0.0375, min_samples =
50).fit(X_principal)
labels1 = db.labels_
```

DBSCAN vs K-Means Clustering

- **When to Use DBSCAN Over K-Means:**

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) and K-Means are both clustering algorithms but differ in how they identify clusters.
Use DBSCAN when the data has **irregular (non-spherical) shapes** or the **number of clusters is unknown.**

- **Summary:**

DBSCAN is ideal for data with complex, uneven cluster structures and noise.
K-Means works best for clean data with well-separated, spherical clusters.

Aspect	DBSCAN	K-Means
Number of Clusters	Not required to be specified	Must be specified beforehand
Cluster Shape	Can form arbitrary-shaped clusters	Assumes spherical or convex clusters
Noise & Outliers	Handles noise and outliers effectively	Sensitive to outliers — can distort clusters
Parameters	Requires two parameters: *eps* and *min_samples*	Requires one parameter: number of clusters (K)

Evaluation Metrics

Category	Metric	Description / Purpose	Formula / Key Concept
Classification Metrics	**Accuracy**	Measures how many predictions were correct out of total predictions. Misleading with imbalanced data.	Accuracy = Correct Predictions / Total Predictions
	Precision	Fraction of correctly predicted positive instances. Useful when False Positives are costly.	Precision = TP / (TP + FP)
	Recall (Sensitivity)	Fraction of actual positives correctly identified. Important when False Negatives are costly.	Recall = TP / (TP + FN)
	F1 Score	Harmonic mean of Precision and Recall. Balances both metrics.	F1 = 2 × (Precision × Recall) / (Precision + Recall)
	Logarithmic Loss (Log Loss)	Measures model uncertainty; penalizes incorrect probabilities. Lower = better.	Log Loss = $-(1/N) \Sigma \Sigma y_{ij} \log(p_{ij})$
	ROC–AUC Score	Evaluates classification thresholds; higher AUC = better model discrimination.	AUC = Area under ROC curve (TPR vs FPR)
	Confusion Matrix	Summarizes predictions into TP, TN, FP, FN. Useful for visualizing errors.	Matrix with Actual vs Predicted values

Evaluation Metrics

Regression Metrics	Description / Purpose	Formula / Key Concept
Mean Absolute Error (MAE)	Average of absolute differences between predicted and actual values.	$MAE = (1/N)\ \Sigma$
Mean Squared Error (MSE)	Average of squared differences; penalizes large errors.	$MSE = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2$
Root Mean Squared Error (RMSE)	Square root of MSE; interpretable in same units as target variable.	$RMSE = \sqrt{(\Sigma\ (y_i - \hat{y}_i)^2 / N)}$
Root Mean Squared Logarithmic Error (RMSLE)	Penalizes underestimation; useful for wide-range data.	$RMSLE = \sqrt{(\Sigma\ (\log(y_i+1) - \log(\hat{y}_i+1))^2 / N)}$
R^2 (Coefficient of Determination)	Proportion of variance explained by the model.	$R^2 = 1 - [\Sigma(y_i-\hat{y}_i)^2 / \Sigma(y_i-\bar{y})^2]$

Evaluation Metrics

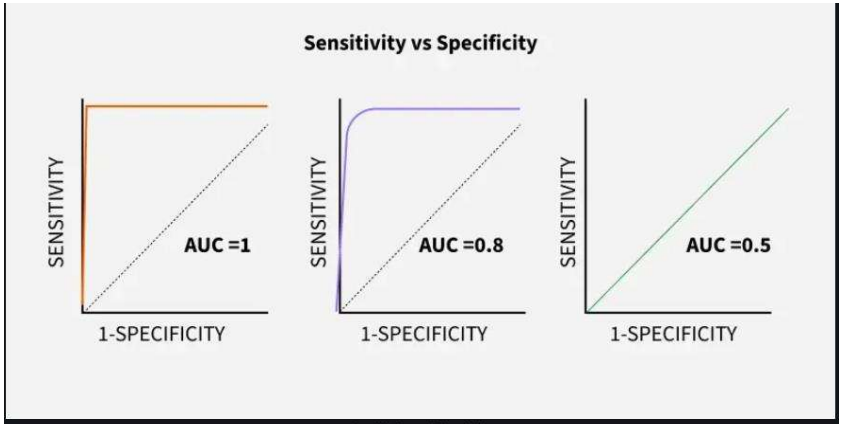
Regression Metrics	Description / Purpose	Formula / Key Concept
Mean Absolute Error (MAE)	Average of absolute differences between predicted and actual values.	$MAE = (1/N)\ \Sigma$
Mean Squared Error (MSE)	Average of squared differences; penalizes large errors.	$MSE = (1/N)\ \Sigma\ (y_i - \hat{y}_i)^2$
Root Mean Squared Error (RMSE)	Square root of MSE; interpretable in same units as target variable.	$RMSE = \sqrt{(\Sigma\ (y_i - \hat{y}_i)^2 / N)}$
Root Mean Squared Logarithmic Error (RMSLE)	Penalizes underestimation; useful for wide-range data.	$RMSLE = \sqrt{(\Sigma\ (\log(y_i+1) - \log(\hat{y}_i+1))^2 / N)}$
R^2 (Coefficient of Determination)	Proportion of variance explained by the model.	$R^2 = 1 - [\Sigma(y_i-\hat{y}_i)^2 / \Sigma(y_i-\bar{y})^2]$

Evaluation metrics

Clustering Metrics	Description / Purpose	Formula / Key Concept
Silhouette Score	Measures how well samples fit their cluster compared to others. Higher = better.	$(b - a) / \max(a, b)$
Davies–Bouldin Index	Measures average similarity between clusters; lower = better separation.	$DBI = (1/N) \Sigma \max_{i \neq j} ((\sigma_i + \sigma_j) / d(c_i, c_j))$

Sensitivity vs Specificity

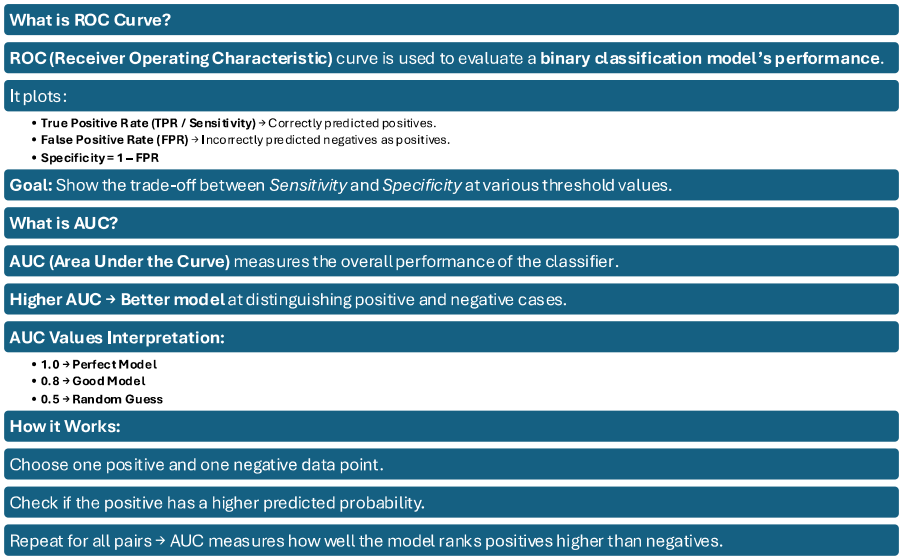
- **Sensitivity (Recall / True Positive Rate):**
- Measures the proportion of *actual positives* correctly identified.
- Focuses on minimizing **False Negatives (FN)**.
- Formula:
  Sensitivity = TP / (TP + FN)
- Example:
  If a fraud detection model correctly detects 90 out of 100 frauds →
  Sensitivity = 90 / (90 + 10) = 0.90
- **High Sensitivity → Few missed positive cases**
- **Specificity (True Negative Rate):**
- Measures the proportion of *actual negatives* correctly identified.
- Focuses on minimizing **False Positives (FP)**.
- Formula:
  Specificity = TN / (TN + FP)
- Example:
  If a model correctly labels 950 of 1000 non-frauds →
  Specificity = 950 / (950 + 50) = 0.95
- **High Specificity → Few false alarms**
- **On ROC Curve:**
- AUC (Area Under Curve) shows the balance between Sensitivity & Specificity.
  - **AUC = 1.0** → Perfect Model
  - **AUC = 0.8** → Good Balance
  - **AUC = 0.5** → Random Guess



Sensitivity and Specificity

Aspect	Sensitivity	Specificity
Definition	Detects actual positives correctly	Detects actual negatives correctly
Focus	True Positives	True Negatives
Error Controlled	False Negatives	False Positives
Useful When	Missing positives is risky (e.g., medical, fraud detection)	False alarms are costly (e.g., credit scoring, spam filters)
Increase When	FN decreases	FP decreases

ROC –AUC Curve



When to Use:

Balanced datasets where both positive and negative classes are important.

When false positives and false negatives have similar cost.

For **imbalanced datasets**, use **Precision–Recall Curve** instead.

Model Performance:

- **High AUC (~1):** Model clearly

BIAS VARIANCE TRADEOFF

Concept

Prediction errors arise from **bias** and **variance**.

A **trade-off** exists between minimizing both — finding the **best regularization constant** balances them.

Goal: **Avoid underfitting** (high bias) and **overfitting** (high variance).

Bias

Difference between model predictions and true values.

High Bias → Underfitting (oversimplified model, e.g., linear fit).

Variance

Measures model sensitivity to training data.

High Variance → Overfitting (too complex model, poor generalization).

Trade-Off

High Bias → Low Variance (simple model)

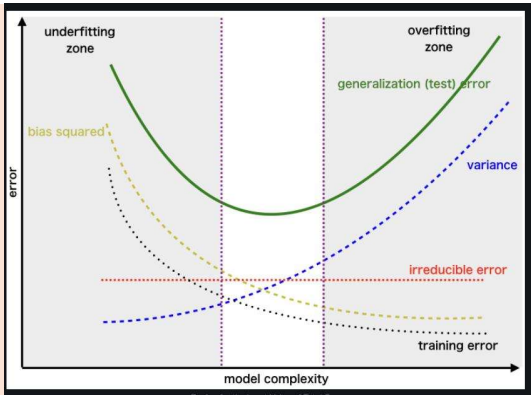
Low Bias → High Variance (complex model)

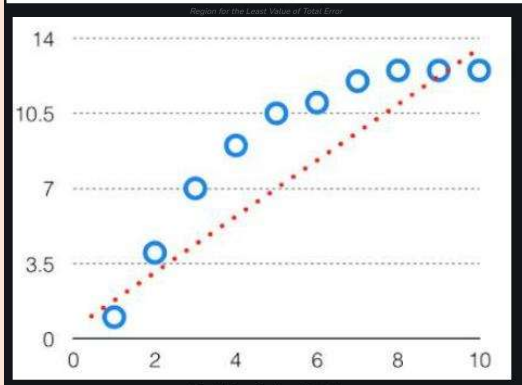
Optimal point minimizes total error:

Total Error=Bias2+Variance+Irreducible Error

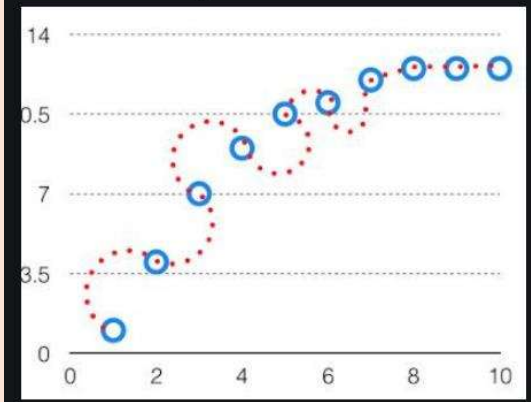
The **best model** lies where training and test errors are both low.

Key Idea:
Find the sweet spot — not too simple, not too complex — for the least total error.





High Bias in the Model



MODEL SELECTION

Why Model Selection Matters

Determines **accuracy, efficiency, and reliability** of predictions.

Poor model → **overfitting** (too complex) or **underfitting** (too simple).

Goal: Learn patterns **effectively** without being too simple or too complex.

Influences **training time**, dataset handling, and computational costs.

Steps in Model Selection

Understand the Problem & Data

- Regression → Predict continuous values (e.g., house prices)
- Classification → Predict categories (e.g., spam vs non-spam)
- Clustering → Group similar data points (e.g., customer segmentation)
- Examine dataset: missing values, categorical/numerical features, distribution

Select Suitable Models

- **Regression:** Linear Regression, Decision Trees, Random Forest, Neural Networks
- **Classification:** Logistic Regression, SVM, k-NN, Neural Networks
- **Clustering:** k-Means, Hierarchical, DBSCAN

Model Evaluation

- Split data: Training set (learn patterns) & Testing set (evaluate)
- Use **k-fold cross-validation** for unbiased evaluation
- Metrics:
  - Regression → MSE, MAE, R^2
  - Classification → Accuracy, Precision, Recall, F1-score

Grid Search

Systematically tries all hyperparameter combinations

Pros: Finds optimal parameters

Cons: Computationally expensive

Random Search

Randomly selects a subset of parameter combinations

Faster than grid search with comparable results

Bayesian Optimization

Uses probability models to **predict best hyperparameters**

Efficient and often better than grid/random search

Cross-Validation Based Selection

Evaluates models on multiple data splits

Averages results → reduces overfitting risk

Ensures good performance on unseen data

Key Takeaways

Proper model selection **improves accuracy, efficiency, and reliability**

Combine **performance metrics, dataset characteristics, and computational resources**

Use **systematic hyperparameter tuning** to find the best model

PRACTICE QUESTIONS Unit III

- Write difference between Classification and regression. (ii) Write five real life application of classification.
- Explain Logistic Regression with example. (ii) Explain Support Vector machine with example.
- Explain Decision Tree with example. (ii) Explain Random Forest with example.
- Explain terms – Classifier, Classification Model, Labels, Features, KNN.
- Explain the complete pipeline for building a machine learning model from scratch. Include data preprocessing, model selection, training, evaluation, and deployment.
- Discuss the importance of splitting data into training, validation, and testing sets. How does each contribute to model development and evaluation?
- Describe various techniques to improve model accuracy and generalization. Include examples like regularization, cross-validation, and feature engineering.
- Explain the working of a Random Forest Classifier. How does it overcome the limitations of a single decision tree?
- Discuss the role of feature importance in Random Forest. How can it be used for feature selection and model interpretation?

NUMERICAL QUESTION Unit III

Datasets have total 20 cats and dog's images. Given that machine learning model trained on Binary classification gives following results. [1] Predicted cat and actual cat are same =10, [2] Predicted not cat and actual is dog = 6 [3] Predicted cat and actual is dog = 3. [4] Predicated not cat and actual is cat =1.

(i) Draw Confusion matrix (ii) Define terms and find value of True Positives, False Positives, True negative, False Negatives (iii) Define terms and find value of Accuracy, Precision, Recall and F1-Score for model.

Datasets have total 35 cats and dog's images. Given that machinelearning model trained on Binary classification gives followingresults. [1] Predicted cat and actual cat are same =10, [2]Predicted not cat and actual is dog = 11 [3] Predicted cat andactual is dog = 5. [4] Predicated not cat and actual is cat =9.

(i) Draw Confusion matrix (ii) Define terms and find value of TruePositives, False Positives, True negative, False Negatives (iii) Defineterms and find value of Accuracy, Precision, Recall and F1-Score formodel.

CASE STUDIES Unit III

Case Study:

A healthcare startup is building a model to predict whether a patient has diabetes (Yes/No). Later, they expand the system to classify patients into one of four disease categories. Explain the difference between binary and multiclass classification and how model design changes accordingly.

- **Binary classification** involves two classes (e.g., diabetic vs non-diabetic). Algorithms like logistic regression or SVM are commonly used.
- **Multiclass classification** involves more than two classes (e.g., diabetes, hypertension, asthma, none).
- Binary models use a single decision boundary, while multiclass models often use one-vs-rest or softmax-based approaches.
- Evaluation metrics shift from binary (accuracy, precision, recall) to macro/micro-averaged scores in multiclass settings.
- Model complexity increases with multiclass tasks, requiring more data and careful handling of class imbalance.

Case Study:

A bank uses decision trees to classify loan applications as approved or rejected, and to predict loan amounts. Explain how decision trees handleboth classification and regression tasks.

- For **classification**, decision trees split data based on criteria like Gini impurity or entropy to separate classes.
- For **regression**, they minimize variance within splits to predict continuous values.
- Trees are built top-down, selecting the best feature at each node.
- They are easy to interpret and visualize, but prone to overfitting.
- Pruning techniques and depth control help improve generalization.

Practice questions Unit IV

Clustering in Machine Learning

Explain the concept of clustering in machine learning. How does it differ from classification, and what are its key applications?

Describe the K-Means clustering algorithm in detail. Include its working steps, objective function, limitations, and methods to choose the optimal number of clusters.

Discuss the DBSCAN algorithm. How does it identify clusters and noise? Compare its strengths and weaknesses with K-Means.

Explain hierarchical clustering. Differentiate between agglomerative and divisive approaches. How is a dendrogram used in this method?

Evaluate clustering models using internal and external metrics. Discuss the role of silhouette score, Davies–Bouldin index, and adjusted Rand index.

Dimensionality Reduction

What is dimensionality reduction? Why is it important in machine learning? Discuss the challenges of working with high-dimensional data.

Explain the Principal Component Analysis (PCA) technique. Include its mathematical foundation, steps, and how it helps in reducing dimensionality.

Describe Linear Discriminant Analysis (LDA). How does it differ from PCA in terms of objective and application?

Compare and contrast feature selection and feature extraction. Provide examples of techniques used for each and discuss their impact on model performance.

Discuss the role of eigenvalues and eigenvectors in PCA. How do they contribute to identifying principal components?

Numerical Questions Unit IV

K-Means Clustering – Solved Example

- Suppose that the data mining task is to cluster points into three clusters,
- where the points are
- *A*1(2, 10), *A*2(2, 5), *A*3(8, 4), *B*1(5, 8), *B*2(7, 5), *B*3(6, 4), *C*1(1, 2), *C*2(4, 9).
- The distance function is Euclidean distance.
- Suppose initially we assign *A*1, *B*1, and *C*1 as the center of each cluster, respectively.

K-Means Clustering Algorithm – Solved Example

- Use K Means clustering to cluster the following data into two groups.
- Data Points: { 2, 4, 10, 12, 3, 20, 30, 11, 25 }
- The distance function used is Euclidean distance.
- Initial cluster centroid are M1 = 4 and M2 = 11.

Numerical Questions Unit IV

DBSCAN Clustering Algorithm Solved Example – 1

- Apply the DBSCAN algorithm to the given data points and
- Create the clusters with
- minPts = 4 and
- epsilon (ε) = 1.9.

Data Points:

P1: (3, 7)	P2: (4, 6)
P3: (5, 5)	P4: (6, 4)
P5: (7, 3)	P6: (6, 2)
P7: (7, 2)	P8: (8, 4)
P9: (3, 3)	P10: (2, 6)
P11: (3, 5)	P12: (2, 4)

DBSCAN Clustering Algorithm Solved Example – 2

Apply the DBSCAN algorithm with similarity threshold of 0.8 (using the similarity matrix) to the given data points and MinPts>=2 (Minimum required points in a cluster) what are core, border and noise (outliers) in the set of points given in table.

	P1	P2	P3	P4	P5
P1	1.00	0.10	0.41	0.55	0.35
P2	0.10	1.00	0.64	0.47	0.98
P3	0.41	0.64	1.00	0.44	0.85
P4	0.55	0.47	0.44	1.00	0.76
P5	0.35	0.98	0.85	0.76	1.00

Numerical Questions Unit IV

Principle Component Analysis – Solved Example

- Given the data in Table, reduce the dimension from 2 to 1 using the Principal Component Analysis (PCA) algorithm.

Feature	Example 1	Example 2	Example 3	Example 4
X_1	4	8	13	7
X_2	11	4	5	14

Linear Discriminant Analysis - Solved Example

- Compute the Linear Discriminant projection for the following two dimensional dataset.

Samples for class ω_1 : $X_1=(x_1,x_2)=\{(4,2),(2,4),(2,3),(3,6),(4,4)\}$

Sample for class ω_2 : $X_2=(x_1,x_2)=\{(9,10),(6,8),(9,5),(8,7),(10,8)\}$

Case Studies Unit IV

- **A hospital wants to group patients based on symptoms to improve diagnosis. How can clustering help in this scenario?**
    - Identifies hidden patterns in patient data
    - Groups similar symptom profiles
    - Aids in early disease detection
    - Supports personalized treatment plans
    - Reduces diagnostic errors
    - Helps allocate medical resources efficiently
    - Enables targeted health campaigns
    - Improves patient monitoring
    - Facilitates research on symptom clusters

- **A travel company wants to categorize destinations based on traveler preferences. Which clustering method is suitable and why?**
    - K-Means for clear, predefined categories
    - DBSCAN for irregular, density-based preferences
    - Handles both popular and niche destinations
    - Reveals hidden travel trends
    - Improves recommendation systems
    - Supports personalized travel packages
    - Enhances user experience
    - Helps in market segmentation
    - Reduces search complexity

Practice questions Unit V

Model Building & Training

Explain the complete process of building a machine learning model from scratch. Include data preprocessing, model selection, training, evaluation, and deployment.

Discuss the importance of splitting data into training and testing sets. How does this impact model performance and generalization?

Describe various techniques used to improve model accuracy and generalization. Include examples such as cross-validation, regularization, and feature engineering.

Random Forest Classifier

Explain the working of a Random Forest Classifier. How does it overcome the limitations of a single decision tree?

Discuss the role of feature importance in Random Forest. How can it be used for feature selection and model interpretation?

Clustering Algorithms

Describe the K-Means clustering algorithm. Include its steps, objective function, limitations, and methods to choose the optimal number of clusters.

Explain the DBSCAN algorithm. How does it identify clusters and noise? Compare its performance with K-Means in handling non-spherical clusters.

Write the steps to implement K-Means and DBSCAN from scratch. Highlight the mathematical intuition behind each algorithm.

Performance Metrics

Define and compare accuracy, precision, recall, sensitivity, and specificity. In which scenarios are each of these metrics most useful?

Explain the concept of confusion matrix. How can it be used to derive various performance metrics for classification models?

Case Studies Unit V

- **Case:** A healthcare startup wants to build a model to predict patient readmission. What steps should they follow to build a machine learning model?
    - Collect historical patient data
    - Clean and preprocess data (handle missing values, encode categories)
    - Perform exploratory data analysis
    - Select relevant features
    - Choose a suitable algorithm (e.g., logistic regression, Random Forest)
    - Split data into training and testing sets
    - Train the model on training data
    - Evaluate using performance metrics
    - Tune hyperparameters for optimization
    - Deploy the model in a hospital system

- **Case:** A bank wants to predict loan defaults. How should they approach model training and testing?
    - Gather customer financial and credit history
    - Preprocess and normalize data
    - Split data into training and testing sets
    - Train model using supervised learning
    - Use cross-validation to avoid overfitting
    - Evaluate using accuracy, precision, recall
    - Adjust model parameters
    - Test on unseen data
    - Compare multiple models
    - Finalize and deploy the best-performing model